



# Team 89 Report of Agile Software Project

## SOULFUL SPOON

**Team Members: Alisha Ayub, Harshikaa Thiagu, Ishita Bhatnagar, Htet Lin Aung**

Link to Videos of our Project: [Soulful Spoon Videos](#)

# 1. Background

## 1.1 Project Overview

We have created a functional website, a platform where users can create a profile, view and upload various recipes, read and leave reviews (likes and comments) and cater their food preferences. The website addresses the difficulties in keeping track of healthier food choices. This provides a representation of how fast paced our life has become, that having an all-in-one platform to provide all these functionalities makes it simpler for users to access. Soulful Spoon aims to create an engaging digital experience for food enthusiasts of all backgrounds.

## 1.2 Outcomes

The primary aim of this project is to provide users with a seamless and interactive food blogging experience by integrating essential functionalities such as user authentication, personalized content, and AI-driven meal planning. The system is designed to enhance user engagement, accessibility, and usability while ensuring an intuitive interface.

As we have finalised the webpage, the project aims to achieve the following key outcomes:

| Key Outcome                                          | Explanations                                                                                                                                                                      |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Authentication and Personalization</b>       | Users should be able to create an account, log in securely, and access a personalized dashboard.                                                                                  |
| <b>Community Engagement</b>                          | Users should be able to interact with recipes by leaving comments, facilitating engagement and discussions within the food blogging community.                                    |
| <b>AI-Integrated Chatbot for Meal Planning</b>       | The implementation of an AI-powered chatbot should allow users to receive personalized meal plans tailored to their dietary needs and preferences.                                |
| <b>Informational Content &amp; Culinary Insights</b> | A dedicated “Tips and Tricks” section should provide users with relevant cooking news, expert advice, and techniques to enhance their culinary knowledge.                         |
| <b>Recipe Management &amp; Filtering</b>             | The platform should enable users to browse a diverse selection of recipes and filter them based on dietary preferences such as halal, vegetarian, and other dietary restrictions. |
| <b>User Account Management</b>                       | A user dashboard page should allow users to                                                                                                                                       |

|                                                  |                                                                                                                                      |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
|                                                  | update their account details, including password, name, email, and profile picture.                                                  |
| <b>Seamless Navigation &amp; User Experience</b> | The platform should be intuitive and user-friendly, ensuring efficient navigation and accessibility across all implemented features. |

### 1.3 Introduction

The aims for this stage of the project are to successfully develop the web application. The team is divided into 1 front end, 2 full stack developers, and 1 back end. The backend team worked on the logic and the algorithms, using SQLite3 and an API, while the team operated with a Svelte template. Hence, making the website work dynamically.

### 1.4 Literature

The increasing global reliance of ultra-processed foods (UPFs) has raised significant health concerns, contributing to rising cases of obesity, diabetes, and cardiovascular diseases. Additionally, the decline in traditional cooking practices has impacted cultural identity and dietary diversity, underscoring the need for innovative solutions. Soulful Spoon aims to address these challenges by fostering community engagement, promoting healthy eating, and preserving culinary traditions instead of eating out.

The literature analysis explores key research areas essential to the development of Soulful Spoon, focusing on:

1. Health Impact of UPFs
2. The Role of Social Media (SM) in the Food Industry
3. Challenges within the Domain

#### 1.4.1 Health Impact of UPFs

Ultra-Processed Foods (UPFs) are industrially manufactured food products composed primarily of refined carbohydrates, unhealthy fats, and artificial additives such as preservatives, sweeteners, colorants, and flavour enhancers. These products typically lack whole food ingredients and are engineered for convenience, long shelf life, and hyper-palatability.

Common examples of UPFs include sugary beverages, processed meats, packaged snacks, ice creams, breakfast cereals, and ready-to-eat meals.

## Frequency of UPF Consumption

### **Case Study 1:**

A study by BMJ 2024 found that direct associations were found between exposure to ultra-processed foods and 32 (71%) health parameters spanning mortality, cancer, and mental, respiratory, cardiovascular, gastrointestinal, and metabolic health outcomes. (BMJ, 2024)

### **Case Study 2:**

Research from Northeastern University's Network Science Institute reveals that 73% of the U.S food supply consists of UPFs, which are, on average, 52% more affordable than minimally processed alternatives (Agostino, n.d.).

These case studies underscore the increasing reliance on UPFs across various demographics and regions, emphasizing the urgent need for public awareness initiatives, and innovative solutions to mitigate the negative health impacts associated with these diets.

### *1.4.2 Role of Social Media in the Food Industry*

As of February 2025, there were 5.56 billion internet users worldwide, which amounted to 67.9 percent of the global population. Of this total, 5.24 billion, or 63.9 percent of the world's population, were social media users. Online sources have the edge among younger consumers, amounting to 38% of Gen Z and millennials (aged 16-38) and 43% of Gen X and baby boomers (aged 39-64) getting their meal inspiration from recipe websites (GWI, 2022).

### **Case Study 1:**

Recent trends highlight the power of social media in food marketing. For example, chaos cakes saw a 45.36% rise in Instagram posts, while pasta chips gained 24.78% more discussions on TikTok, showcasing how viral content can dictate food preferences. Additionally, 90% of consumers research restaurants online before dining, with 50% basing their choices on social media influence. This shift underscores how social media has evolved from a simple content-sharing platform to a critical decision-making tool that influences global food culture and consumer engagement according to the 2025 GWI survey.



## Case Study 2:

A viral TikTok on September 21 featuring a food blogger, Emily Mariko, smashing up cooked salmon, and topping it with leftover sushi rice, then putting it in a microwave. Then, she topped it off with a mix of soy sauce, sriracha and Kewpie mayo. This trend went viral, viewed by 6.9 million people, causing an international shortage of Kewpie Mayo in local supermarkets, demonstrating the platform's ability to drive sudden demand surges (Eater, 2021).

The significant impact of social media on food choices presents an opportunity for Soulful Spoon to leverage these platforms to promote healthier eating habits. By adopting engaging digital marketing strategies, collaborating with health-conscious influencers, and creating viral recipe trends, the platform can encourage nutritious food choices while preserving culinary diversity.

### 1.4.3 Domain-Specific Challenges

#### a) Competitive Market

*Challenge:* The food and lifestyle sector is dominated by established platforms like Pinch of Yum and Food Network, making it difficult for new entrants to gain visibility.

*Solution:* Position Soulful Spoon as a social-media-integrated food platform, offering personalized meal plans and community-driven content to differentiate itself.

#### b) Audience Engagement

*Challenge:* With teenagers spending an average of 143 minutes daily on social media (Geuens, 2024), capturing their attention requires compelling and interactive content.

*Solution:* Develop visually appealing, culturally relevant content and utilize trending formats, influencer collaborations, and user-generated content to enhance engagement.

#### c) Addressing Diverse Dietary Needs

*Challenge:* The global shift toward specialized diets (vegan, gluten-free, low-sugar) is not fully accommodated by mainstream food platforms (WHO, 2024).

*Solution:* Implement recipe filters to cater to diverse dietary preferences and enhance accessibility to suitable meal options.

#### d) Balancing Health and Affordability

*Challenge:* Healthier diets are, on average, twice as expensive as processed alternatives, limiting accessibility for 35% of the global population (Concern Worldwide, 2024).

*Solution:* Introduce a "Budget Bites" section, featuring affordable, nutritious recipes using locally available ingredients.

These challenges reflect current industry trends and consumer concerns, ensuring Soulful Spoon remains innovative and adaptable.

## 2. Planning and Research

| Pages             | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Remarks                                                                                                                                                                                                                 |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Login Page        | <b>Content</b> <ul style="list-style-type: none"> <li>Include a "Sign Up Now!" button</li> </ul> <b>Features</b> <ul style="list-style-type: none"> <li>OAuth integration (Google, Facebook)</li> <li>Role-based access (users, admins) - logged in / not logged in</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                         |
| Signup Page       | <ul style="list-style-type: none"> <li>Include "Login" button</li> </ul> <b>Features</b> <ul style="list-style-type: none"> <li>OAuth integration (Google, Facebook)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                         |
| Home Page         | <b>Content</b> <ul style="list-style-type: none"> <li>Main banner</li> <li>Search bar</li> <li>Healthy recipes (with filters)</li> <li>Dietary specific recipes (with filters)</li> <li>Recent recipes (with filters)</li> <li>Trending recipes (with filters)</li> <li>Latest on cooking (video, articles etc)</li> <li>Sign up for newsletter section</li> </ul> <b>Note</b> <ul style="list-style-type: none"> <li>Add more video recipes (multimedia integration)</li> <li>More personalisation</li> </ul> <b>Features</b> <ul style="list-style-type: none"> <li>AI integration (personalised meal plans, recipes curated for each individual, chatbots)</li> </ul> | <a href="https://pinchofyum.com/">https://pinchofyum.com/</a><br><a href="https://www.foodnetwork.com/">https://www.foodnetwork.com/</a><br><a href="https://dish.co.nz/tips-tricks">https://dish.co.nz/tips-tricks</a> |
| Recipe Page       | <b>Content (Categorised)</b> <ul style="list-style-type: none"> <li>Main banner</li> <li>Search bar</li> <li>Filtering options</li> <li>"Breakfast", "Lunch", "Dinner", "Quick", "Easy"</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                         |
| Individual Recipe | <b>Content</b> <ul style="list-style-type: none"> <li>Main banner</li> <li>Author name and profile picture</li> <li>Published date and time</li> <li>Recipe Image</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                         |

|                         |                                                                                                                                                                                                                                         |                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
|                         | <ul style="list-style-type: none"> <li>• Recipe name</li> <li>• Recipe description</li> <li>• Actual recipe</li> <li>• No.of likes, saves, comments?</li> </ul>                                                                         |                                                                       |
| Article Publishing Page | <b>Content</b> <ul style="list-style-type: none"> <li>• Recipe image</li> <li>• Recipe title</li> <li>• Recipe description</li> <li>• Actual recipe</li> <li>• Tags (for filtering)</li> <li>• Comments (enabled, disabled)?</li> </ul> | Frontend & Logic Done<br>Billy has to check logic and do the backend. |
| Tips and Tricks         | <b>Content</b> <ul style="list-style-type: none"> <li>• News on cooking</li> <li>• Tips on cooking</li> <li>• Articles, videos (multimedia)</li> <li>• </li> </ul>                                                                      |                                                                       |
| Author Profile Page     | <b>Content</b> <ul style="list-style-type: none"> <li>• Author name and profile picture</li> <li>• About me, bio</li> <li>• Recipes uploaded</li> </ul>                                                                                 |                                                                       |
| User Dashboard          | <b>Content</b> <ul style="list-style-type: none"> <li>• User name, bio, email/number - settings page</li> <li>• Preferences page (website color and text size)</li> </ul>                                                               |                                                                       |
| About us                | <b>Content</b> <ul style="list-style-type: none"> <li>• Contact us (form)</li> <li>• Feedback form</li> </ul>                                                                                                                           |                                                                       |

## 2.1 Planning

### Frontend:

We initially considered using Tailwind CSS for styling due to its utility-first approach and responsiveness. However, we encountered compatibility issues with the Svelte framework, which led us to switch to traditional CSS. This approach allows for greater control over styling while ensuring the website remains visually appealing and responsive across different devices. Svelte and SvelteKit were selected as the primary frontend framework due to its lightweight nature and the ability to compile at build time, reducing runtime overhead.

### Backend:

To develop, we implemented a secure and scalable infrastructure. SQLite3 was chosen as the database due to its lightweight nature and suitability for recipe-based applications that require using JSON web Tokens (JWT) and Firebase Authentication, allowing for secure and efficient session management. Express.js was utilized to create RESTful APIs that manage user authentication, recipe management, and interactions, while Bcrypt.js was employed for password hashing to enhance security. Additionally, authentication middleware was incorporated to ensure secure access control for user-related functionalities.

Our research identified that a structured navigation system with dropdowns improves user accessibility. User authentication (login/sign) is critical for personalised interactions. A filtering system enhances user experience by allowing them to find specific dietary or meal-type recipes efficiently. A user-friendly article publishing system increases engagement and community contributions.

- Market analysis: Studied existing recipe-sharing platform.
- Technology: Choose Svelte, Firebase, and Traditional CSS for frontend development and intuitive UI.
- Competitor Benchmarking: Analyzed features from platforms like Dish, Pinch of Yum, and FoodNetwork.

### Final Refinements & Lesson Learned

#### Key achievements

- Successfully implemented authentication, filtering and AI chatbot.
- Built a scalable and responsive UI
- Stored and fetched user-generated content from SQLite3.

Competitor analysis played a crucial role in shaping the features of our platform. We examined popular recipe-sharing websites to identify gaps in personalisation and AI-driven recommendation. One key finding was that many existing platforms lacked a robust filtering system for dietary preferences, which we addressed by implementing a structured filtering mechanism.

```
13     },
14     "devDependencies": {
15         "@eslint/compat": "^1.2.5",
16         "@eslint/js": "^9.18.0",
17         "@sveltejs/adapter-auto": "^4.0.0",
18         "@sveltejs/kit": "^2.16.0",
19         "@sveltejs/vite-plugin-svelte": "^5.0.0",
20         "autoprefixer": "^10.4.20",
21         "eslint": "^9.18.0",
22         "eslint-config-prettier": "^10.0.1",
23         "eslint-plugin-svelte": "^2.46.1",
24         "globals": "^15.14.0",
25         "postcss": "^8.5.2",
26         "prettier": "^3.4.2",
27         "prettier-plugin-svelte": "^3.3.3",
28         "svelte": "^5.0.0",
29         "tailwindcss": "^4.0.6",
30         "vite": "^6.0.0"
31     },
32     "dependencies": {
33         "@botpress/webchat": "2.3",
34         "@tinymce/tinymce-svelte": "^3.0.0",
35         "firebase": "^11.3.1"
36     }
37 }
```

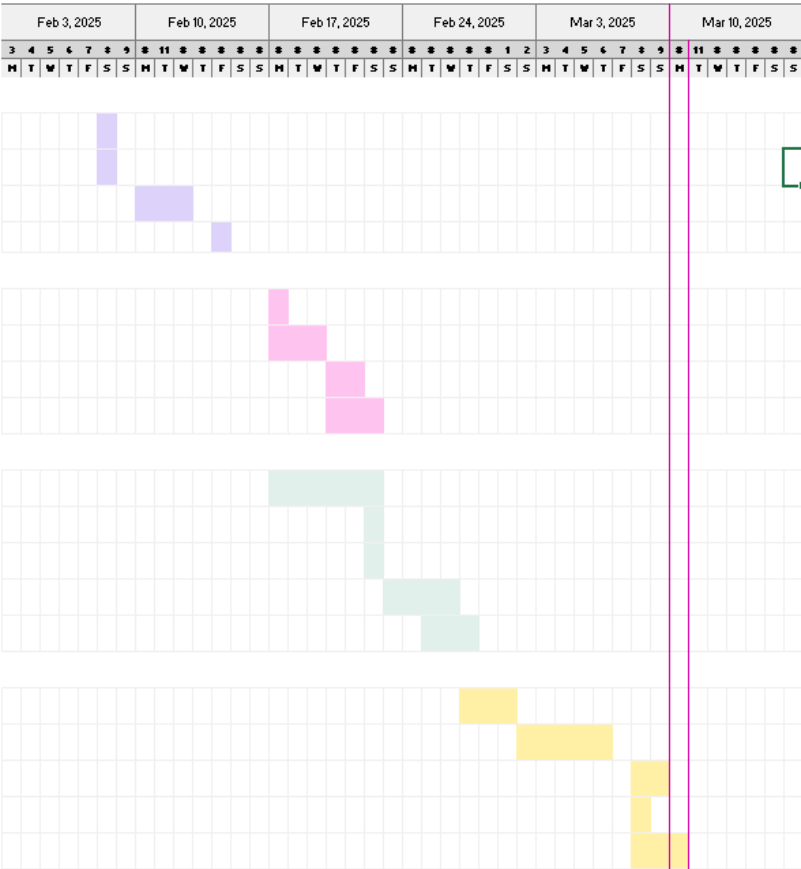
Team 89

Project start: Sat, 2/8/2025

Display week: 1

SIMPLE GANTT CHART by Vertex42.com  
https://www.vertex42.com/ExcelTemplates/simple-gantt-chart.html

| TASK                                    | PROGRESS | START   | END     |
|-----------------------------------------|----------|---------|---------|
| Initiation                              |          |         |         |
| Define goals                            | 100%     | 2/8/25  | 2/8/25  |
| Ideas                                   | 100%     | 2/8/25  | 2/8/25  |
| Consult with Lecturer about idea        | 100%     | 2/10/25 | 2/12/25 |
| Planning on starting                    | 100%     | 2/14/25 | 2/14/25 |
| Planning and design                     |          |         |         |
| Assign tasks                            | 100%     | 2/17/25 | 2/17/25 |
| Frontend UI & backend                   | 100%     | 2/17/25 | 2/19/25 |
| Create Survey                           | 100%     | 2/20/25 | 2/21/25 |
| first UI Iteration                      | 100%     | 2/20/25 | 2/22/25 |
| Execution                               |          |         |         |
| Execute tasks know which pages to m     | 100%     | 2/17/25 | 2/22/25 |
| Monitor progress + Survey               | 100%     | 2/22/25 | 2/22/25 |
| chatbot api integration                 | 100%     | 2/22/25 | 2/22/25 |
| second iteration of UI and backend      | 100%     | 2/23/25 | 2/26/25 |
| backend tables, api, authentication     | 100%     | 2/25/25 | 2/27/25 |
| Evaluation                              |          |         |         |
| making sure backend works with front    | 100%     | 2/27/25 | 3/1/25  |
| flow of website homepage, login, signou | 100%     | 3/2/25  | 3/6/25  |
| future improvments                      | 100%     | 3/8/25  | 3/9/25  |
| Finalize the backend                    | 100%     | 3/8/25  | 3/8/25  |
| Evaluation + report                     | 100%     | 3/8/25  | 3/10/25 |



Gantt Chart of our timelines and process.

### 3. Prototyping and Iteration

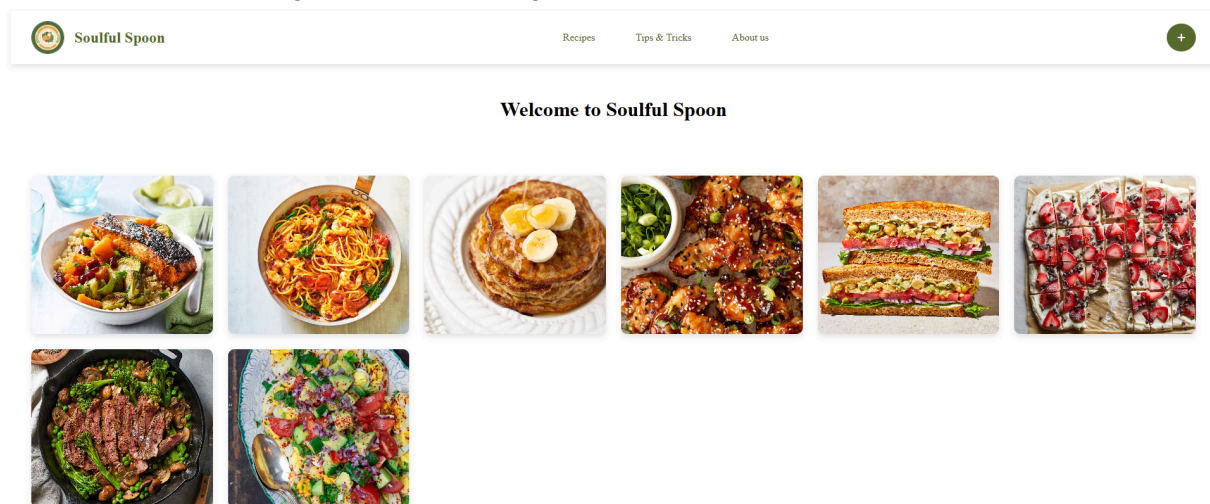
Link to Videos of our Project: [Soulful Spoon Videos](#)

Initial concept and design justification:

The project was followed by an iterative approach, progressing from initial wireframes to working prototyping. The iterations were based on continuous feedback and systematic evaluation.

First Iteration(static structure & Initial UI):

1. Basic HTML structure with a static navbar and content placeholder.
2. Initial layout of homepage with static recipe listing.
3. Initial CSS layout for homepage styling
4. Evaluated navigation flow and page responsiveness across different screens

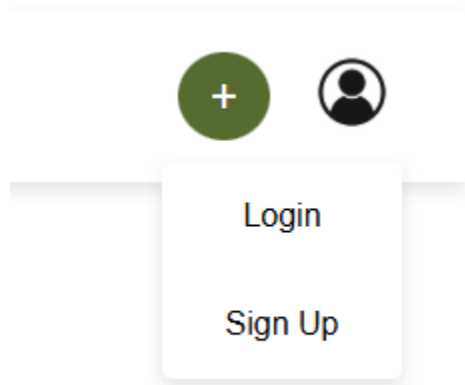


Second Iteration(Interactive feature & Authentication):

1. Adding dynamic filtering for recipes using radio buttons.
2. Implementing a basic authentication system (Login/Signup) with database integration.
3. Integrated a prototype article publishing system.

#### Third Iteration:

1. Improved CSS styling to enhance user experience.
2. Optimized dropdown menus for smoother navigation.
3. Implemented user authentication (Signup/Login) using Firebase for managing user sessions. Resolved login/signup dropdown disappearing issue by adjusting CSS visibility properties.
4. Linked “+” button to navigate to the article publishing page.
5. Integrated embedded YouTube video support for cooking tips.



#### Final iteration: (Performance Optimization)

1. Dynamic Recipe filtering and search bar
2. Authentication system
3. Article publishing system when a user has signed up and logged in
4. YouTube video integration to help users learn cooking techniques
5. Smooth scrolling: users can navigate smoothly

#### Workflow:

A structured approach was used to allocate time and resources efficiently. We utilized tools such as Figma for wireframing and Visual Studio Code for development. The backend was developed using SQLite database management and Svelte framework for frontend interactions. A structured workflow ensured new features were developed and tested in individual branches before merging. This reduces errors and allows iterative improvements.

#### Version control and collaboration:

To ensure efficient version control and collaborative development, our team utilized Git and GitHub throughout the project lifecycle. The repository was maintained in Git, which allowed seamless tracking of changes, branching for feature development, resolving conflicts during integration. We followed a structured branching strategy where different features, such as login,

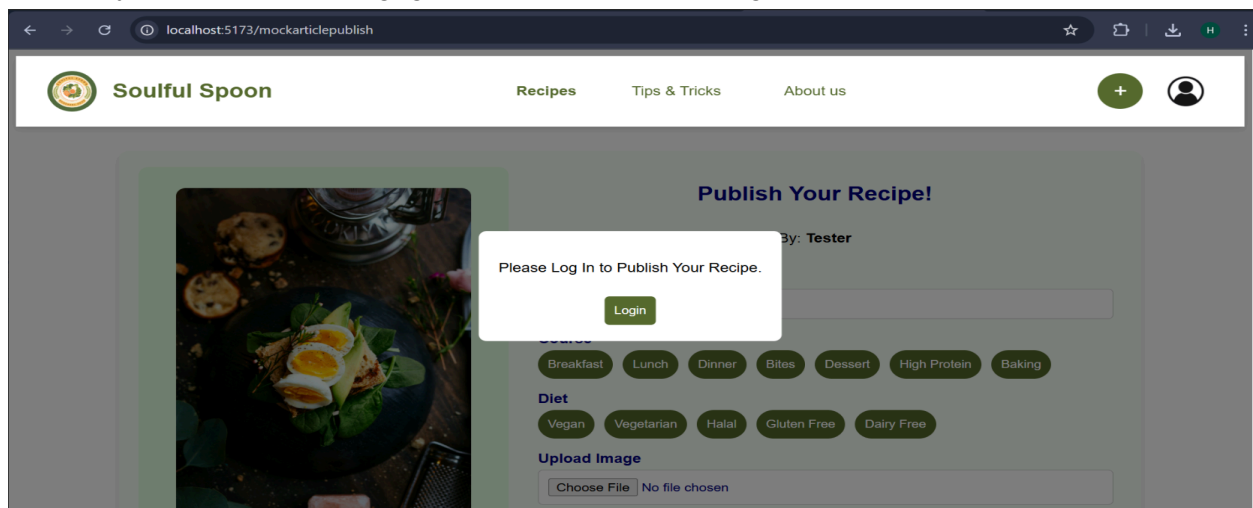


signup, homepage, and chatbot API integration, were developed in separate branches of each team member's name before merging into the main branch.

We made commits to ensure that each change was well-documented. Git messages were descriptive and detailed, such as “added homepage nav bar” or “added login and signup functionality”. To resolve any conflicts, during collaborative work, merge conflicts occasionally occur when team members modify the same file. Through communication and using Git merge strategies, we ensured that no critical updates were lost.

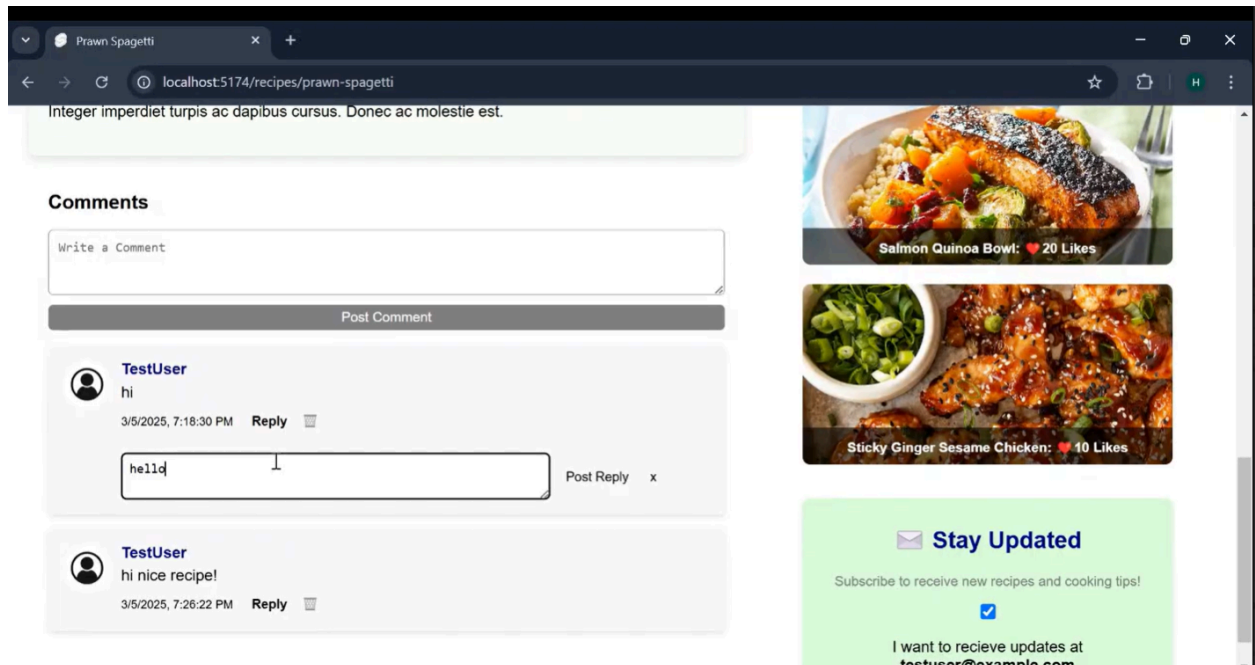
We added the article publishing system:

- To encourage user-generated content, allowing experienced cooks to share their own recipes, kitchen tips, and food experiences.
- It helps to diversify content beyond just pre-listed recipes, making the platform more dynamic. It boosts engagement and search rankings.



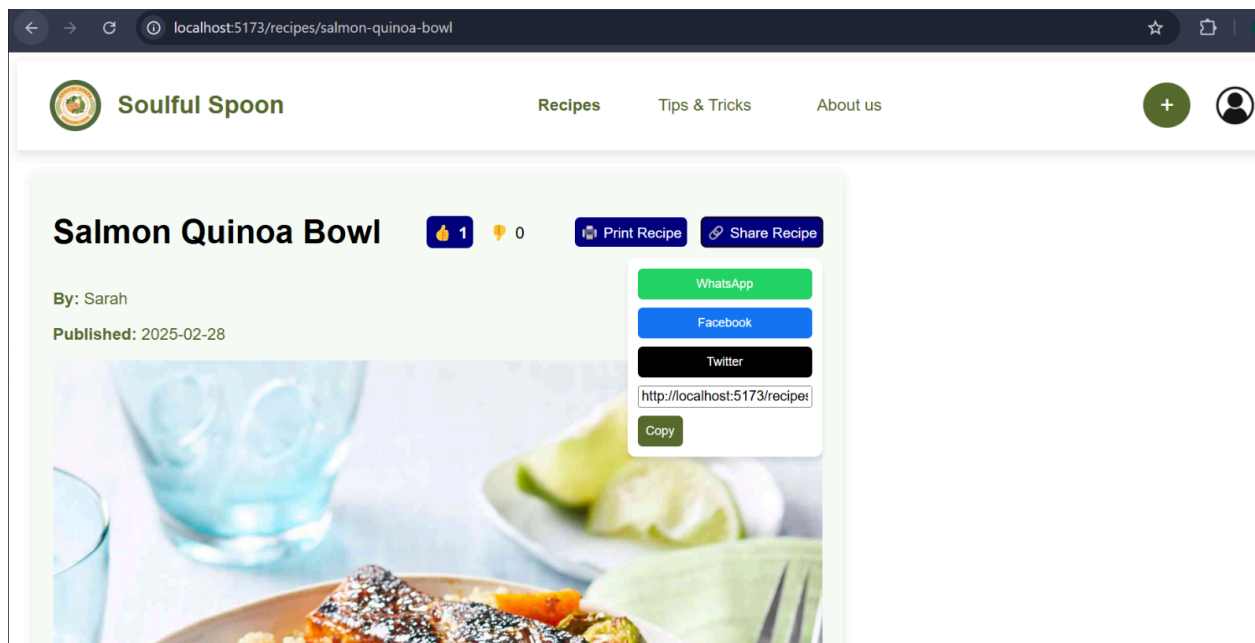
Comments Section:

- Allow users to share feedback, modifications, and personal experiences about recipes.
- Encourages social proof, where users can see others' positive experiences before trying a new recipe.
- Enhances community building, where users can ask for clarifications or share cooking tips. It keeps users on the platform longer.



#### Like System:

- Helps users identify popular recipes at a glance.
- Provides positive reinforcement to content creators who see their recipes appreciated.



#### Messaging System (About us page)

- Enables direct communication between users and platform admins.
- Allows users to submit feedback, collaboration requests, or inquiries regarding content or site features.

- Enhances support and trust, as users can report issues or seek clarifications.



### Get in Touch

Have questions, suggestions, or just want to say hello? We'd look to hear from you! Feel free to reach us to us using the contact details below.



**Email Us**

soufulspoon89@gmail.com



**Our Location**

Singapore

### Send Us a Message

Your Name

Your Email

Your Message

Send Message

Message system code:

```

let name = "";
let email = "";
let message = "";
let successMessage = "";
let errorMessage = "";

async function sendMessage(event) {
  event.preventDefault();

  const response = await fetch("http://localhost:5173/api/messages/send-message", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({name, email, message})
  });

  const data = await response.json();
  if(response.ok) {
    successMessage = "Message sent successfully!";
    name = email = message = "";
  } else {
    errorMessage = data.error || "Failed to send message,";
  }
}
</script>

<div>
  <script src="https://cdn.botpress.cloud/webchat/v2.2/inject.js"></script>
  <script src="https://files.bpcontent.cloud/2025/02/25/06/20250225063934-HGA74MAU.js"></script>
</div>

```

```

<section class="contact-section">
  <div class="contact-container">
    <h2>📞 Get in Touch</h2>
    <p>Have questions, suggestions, or just want to say hello? We'd look to hear from you!
      Feel free to reach us to us using the contact details below.
    </p>
    <div class="contact-info">
      <div class="contact-item">
        <h2>✉ Email Us</h2>
        <p><a href="mailto:soulfulspoon89@gmail.com">soulfulspoon89@gmail.com</a></p>
      </div>
      <div class="contact-item">
        <h3>📍 Our Location</h3>
        <p>Singapore</p>
      </div>
    </div>
  </div>
  <div class="contact-form">
    <h3>💬 Send Us a Message</h3>
    <form on:submit={sendMessage}>
      <input type="text" bind:value={name} placeholder="Your Name" required />
      <input type="email" bind:value={email} placeholder="Your Email" required />
      <textarea bind:value={message} placeholder="Your Message" rows="5" required></textarea>
      <button type="submit">Send Message</button>
      {#if successMessage}
      <p class="success-message">{successMessage}</p>
      {/if}
      {#if errorMessage}
      <p class="error-message">{errorMessage}</p>
      {/if}
    </form>
  </div>
</section>

```

Filter and Search bar:

```

<script>
  import {auth, logout} from '$lib/firebase';
  import { onMount } from 'svelte'; //added
  import { applyAction } from '$app/forms';

  let searchQuery = "";
  let selectedCourse = "";
  let selectedDiet = "";
  let isAuthenticated = false;
  let user = null;
  // let isDarkMode = false; //light mode default

  // const savedTheme = localStorage.getItem("theme");
  // if (savedTheme) {
  //   isDarkMode = savedTheme === "dark";
  //   applyTheme();
  // }

  // //toggle light and dark mode
  // function toggleTheme() {
  //   isDarkMode = !isDarkMode;
  //   localStorage.setItem("theme", isDarkMode ? "dark" : "light");
  //   applyTheme();
  // }

  // //applying theme
  // function applyTheme() {
  //   document.documentElement.classList.toggle("dark-mode", isDarkMode);
  // }

  function clearFilters() {
    searchQuery = "";
    selectedCourse = "";
    selectedDiet = "";
    filteredImages = images;

    document.querySelectorAll('input[name="course"]').forEach((el) => el.checked = false);
    document.querySelectorAll('input[name="diet"]').forEach((el) => el.checked = false);
  }

  onMount (() => {
    auth.onAuthStateChanged((u) => {
      if (u) {
        isAuthenticated = true;
        user = u;
      } else {
        isAuthenticated = false;
      }
    })
  })

```

```

<!-- search bar -->
<div class="search-filter-container">

  <!-- Filters -->
  <div class="filter-box">
    <button class="filter-button">
      Filter Options &</button>

    <div class="filter-dropdown">
      <h4>Courses</h4>
      <ul>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="breakfast" on:change={handleSearch}/> <label for="breakfast">Breakfast</label></li>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="lunch" on:change={handleSearch}/> <label for="lunch">Lunch</label></li>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="dinner" on:change={handleSearch}/> <label for="dinner">Dinner</label></li>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="bites" on:change={handleSearch}/> <label for="bites">Bites</label></li>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="highprotein" on:change={handleSearch}/> <label for="highProtein">High Protein</label></li>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="dessert" on:change={handleSearch}/> <label for="dessert">Dessert</label></li>
        <li><input type="radio" name="course" bind:group={selectedCourse} value="baking" on:change={handleSearch}/> <label for="baking">Baking</label></li>
      </ul>
      <h4>Dietary Requirements</h4>
      <ul>
        <li><input type="radio" name="diet" bind:group={selectedDiet} value="vegan" on:change={handleSearch}/> <label for="vegan">Vegan</label></li>
        <li><input type="radio" name="diet" bind:group={selectedDiet} value="vegetarian" on:change={handleSearch}/> <label for="vegetarian">Vegetarian</label></li>
        <li><input type="radio" name="diet" bind:group={selectedDiet} value="halal" on:change={handleSearch}/> <label for="halal">Halal</label></li>
        <li><input type="radio" name="diet" bind:group={selectedDiet} value="dairyfree" on:change={handleSearch}/> <label for="dairyFree">Dairy Free</label></li>
        <li><input type="radio" name="diet" bind:group={selectedDiet} value="glutenfree" on:change={handleSearch}/> <label for="glutenFree">Gluten Free</label></li>
      </ul>
      <!-- clear button -->
      <button class="clear-button" on:click={clearFilters}>Clear All</button>
    </div>
  </div>
  <div class="search-container">
    <input
      type="text"
      bind:value={searchQuery}
      on:input={handleSearch}
      placeholder="Search recipes..."
      class="search-input"
    />
    <button class="search-button" on:click={handleSearch}>⏎</button>
  </div>
</div>

```

```

let images = [
  {src: "PastaSpaghetti", text: "Pasta Spaghetti", course: ["dinner"], diet: ["halal", "high-protein"]},
  {src: "BananaOatmealPancakes", text: "Banana Oatmeal Pancakes", course: ["breakfast"], diet: ["halal", "high-protein", "vegetarian", "dairy-free"]},
  {src: "StickyGingerSesameChicken", text: "Sticky Ginger Sesame Chicken", course: ["lunch", "dinner"], diet: ["halal", "high-protein"]},
  {src: "ChickpeaSaladSandwich", text: "Chickpea Salad Sandwich", course: ["dinner", "lunch"], diet: ["halal", "vegan", "dairy-free"]},
  {src: "StrawberryYoghurtBites", text: "Strawberry Yoghurt Bites", course: ["bites"], diet: ["halal", "vegetarian"]},
  {src: "SteakWithMushroomSauce", text: "Steak With Mushroom Sauce", course: ["dinner", "lunch"], diet: ["high-protein", "halal"]},
  {src: "EggSalad", text: "Egg Salad", course: ["breakfast"], diet: ["dairy-free", "gluten-free", "halal"]}
];

//reactive variables for filtered images
let filteredImages = images;

//function to handle search
function handleSearch() {
  filteredImages = images.filter(image =>
    (searchQuery === "" ||
      image.text.toLowerCase().includes(searchQuery.toLowerCase()) ||
      image.course.some(c => c.toLowerCase().includes(searchQuery.toLowerCase())) ||
      image.diet.some(d => d.toLowerCase().includes(searchQuery.toLowerCase()))
    ) &&
    (selectedCourse === "" || image.course.includes(selectedCourse)) &&
    (selectedDiet === "" || image.diet.includes(selectedDiet))
  );
}

onMount(() => {
  filteredImages = images;
});

//added by Harshikaa to show user-published and default recipes. have to replace with backend database
onMount(() => {
  if(typeof localStorage !== "undefined"){
    let savedRecipes = JSON.parse(localStorage.getItem("recipes")) || [];

    images = [
      ...images,
      ...savedRecipes.map(recipe => ({
        src: recipe.image_path || "/src/lib/components/images/publish-article.jpg",
        text: recipe.title
      }))
    ];
  }
});

```

## 4. Design and System Development

## **4.1 System Overview**

Svelte framework serves as its lightweight technological trending features alongside SvelteKit for encompassing its fast-paced usability for empowering frontend templates and layouts technology with Svelte syntax enhancing HTML and stylesheet scripting which is more comprehensible among other JavaScript frameworks for instance VUE framework and reactive nature, which ensures smooth user experience. SQLite was selected as it is efficient for table handling as its easiest and swift development, which will be directly linked with data access objects JavaScript logics. Additionally, Bcrypt was integrated to enhance security for password hashing. Node.js enabled whole backend technology with RESTful api and Express.js which mounted the whole project as running environment materials for server side rendering objectives.

## **4.2 Technology Stack**

### **4.2.1 Client Side Elucidation**

Since the mystery of Node.js perks work the flow of virtual running and linking tools of server side rendering which joined the conceptual logic of data fetching for client side codes files of svelte components which are specifically applied to develop the viewing pages and layouts of the web pages. For the layouts and templates, separate stylesheets format is utilized with CSS scripting as well as inline style coding inside Svelte components which is facilitated both HTML and CSS components especially designing the form structure and layouts with Svelte logic which mainly grasp the usage of JavaScript logic with different syntax from Vanilla JavaScript. Integrating the usage of link API URL routes handling allows public environment variables to be defined at build time and API base url is embedded during the compilation where the Base URL is the root of all API endpoints used by multiple HTTP requests. Moreover, all the constants of functions from Svelte like login, signup, likes, comments, and messages are defined at each respective endpoint.

### **4.2.2 Server Side Elucidation**

The technology of Node.js environment empower by conveying the compilation of server side technology as well as some of reliable npm dependencies for instance bcrypt.js for password hashing and salting where user's password cannot be stored in the database as simple string format and it hash and salt all the password filled out by users inside the database as well as yup modules for user's data validation. RESTful API plays the keys for enabling endpoints determination for all of the functionality features for blogging website application and SQLite3 version for the convenience of database utilization of all the objective entities and attributes.

## **4.3 System Architecture**

The backend follows an architecture based on microservices which ensures modular development alongside easy scalability. Since all the backend services engage in communication with API endpoints, the packages of middleware and authentication combine their unique features for enabling the easiest usability of the whole backend Node.js features.

**API Design:** RESTful API and Express.js routers help splitting the server logic codes across multiple files for project structure maintainability, where “import” and “export” functions are used for modularizing all the routes ends. REST API ensures all API paths representing various objects with stateless communication where each request needs to contain all necessary CRUD methods and information, which is uniformed interfaces. Alongside this, HTTP response codes, for instance “200 OK” and “201 Created,” play successful responses where all can be tested via Postman software.

**Middleware for Code Optimization:** The thing middleware helps with avoiding the duplication of code by pre-processing requests, for instance, helps to check if a user exists before compiling routes.

**Data Validation:** Yup modules function to prevent the invalid data from being processed, which is considered as a schema package for runtime parsing of data. And then, transform into matching values.

**Data Loading Cookies and Authentication:** Cookies enhance the functionality to store key-value pairs to each request sent by the browser. The usage of cookies can be different on each perspective, whereas it maintains user authentication of login sessions, stores temporary user data, as cookies are persistent for all kinds of authentications across website visits. CORS configuration enables using the cookies with data fetching, where the backend must specify all the origins.

**Security Considerations:** JSON Web Tokens manage the creation and validation of tokens, where it verifies a single JWT and extracts the user credentials by creating a token with a 24-hour expiry. As aforementioned, bcrypt.js from the npm package also plays the good point of security and privacy concerns for the database accessibility, validate user password data afterwards, hash all the entries and then insert the new user’s data under the user table. The function works by defining the number of salt rounds, where the number of times the password is hashed before storing it. After that, it processes the password matching validation function.

**Database Design:** Without a Database Management System, a full-stack application isn’t an ideal deliverable for data sources handling data redundancy and inconsistency. For the backend side, database.js handles the database connection and initialization using DBMS software and for this project, SQLite is applied for its ease and simplicity with fast deployment. The purpose of database.js involves linking of SQL queries and loading the SQL schema where the script ensures that the database is properly initialized before usage of SQL queries for data insertion.

## 4.4 System Development Process

The whole project follows the workflow of Agile Methodology with three-week sprints allowing the initial and iterative development as well as concluded with users feedback. Each sprint begins with planning sessions, where tasks are prioritized based on business requirements and user needs. Throughout the sprint, the team engages in daily stand-ups to ensure smooth communication, address challenges, and maintain alignment.

At the end of each sprint, a review and retrospective is conducted, allowing the team to showcase progress, gather valuable user feedback, and incorporate improvements in the next iteration. This approach fosters continuous enhancement, adaptability to change, and user-centric development, ensuring the final product aligns closely with stakeholder expectations.

### Full-Stack Development Process and Testing:

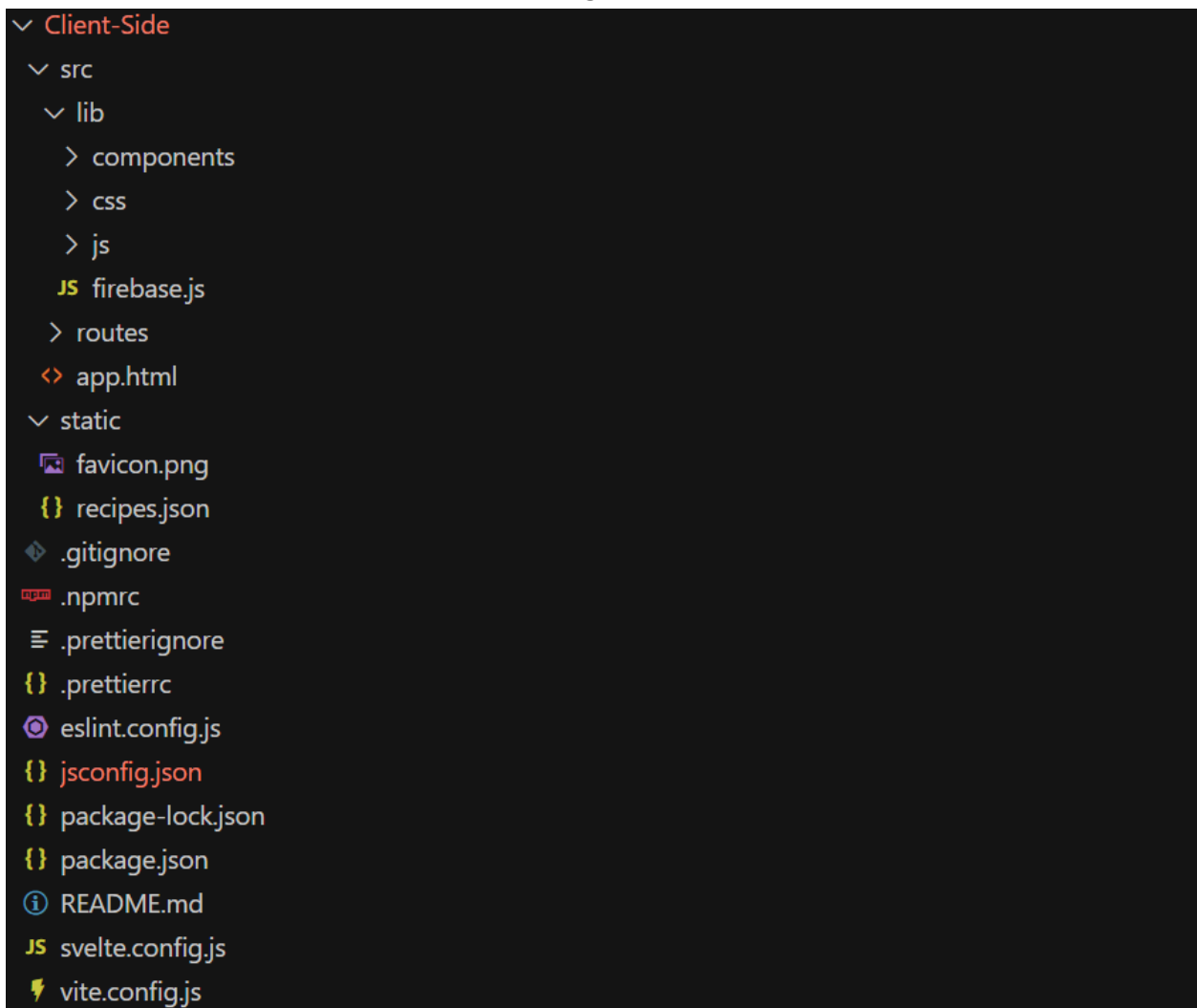
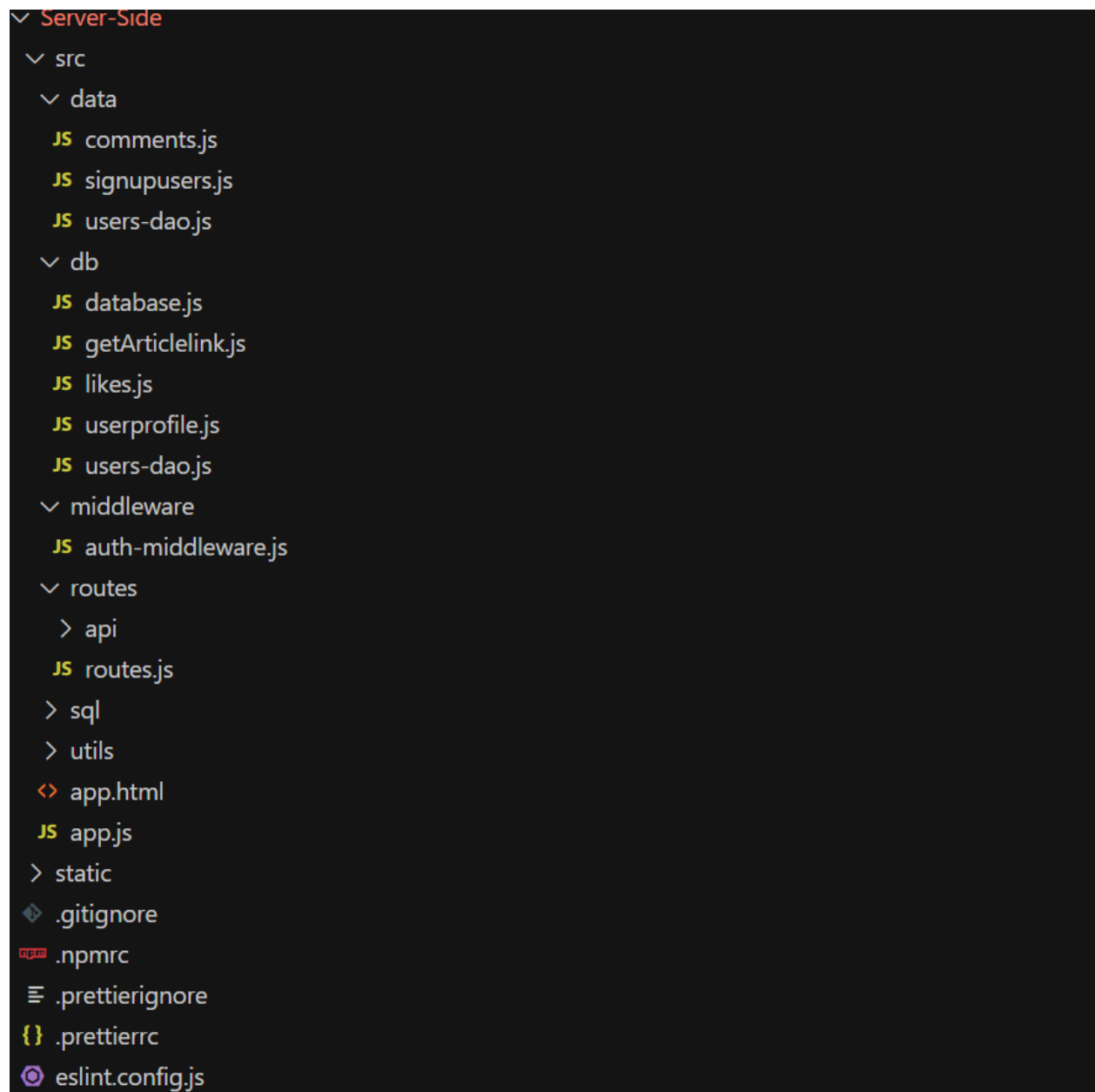


Figure: Frontend directory structure



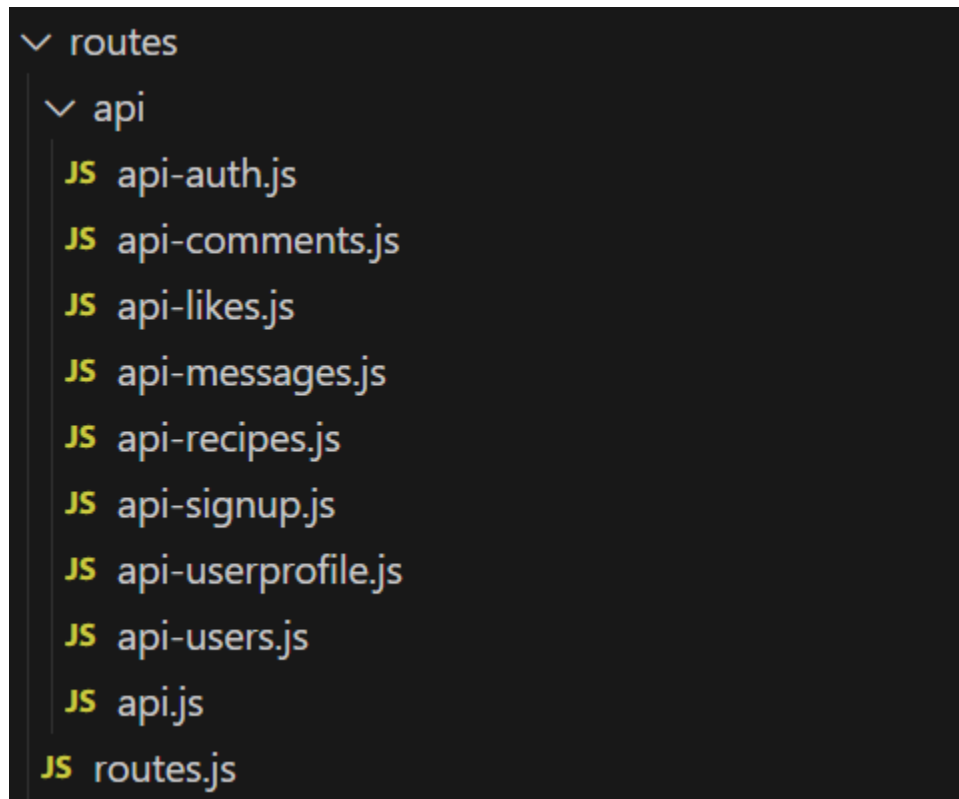


*Figure: Backend directory structure*

The whole full-stack structure displays the systematic directory management of multiple components of files. Frontend has the structure composed mainly of Svelte components under “src/lib/components” that mainly handle the JavaScript logic of web pages, CSS for layout design, and API routes handling logic under “src/lib/js”. Under the “src/routes” directory, the Svelte logics are used to unveil the page layout files from the components folder as separate and single web pages.

Backend consists of logic for handling API requests, authentication, database operations, and routing. Under the “src/data” directory, files are responsible for handling database operations related to users, comments, and signup logic by managing each related database query. Data

Access Object file includes the functions of fetching user details by credentials, checking login credentials by verifying hashed passwords, and retrieving user-related data. And, API calls from the frontend use users-dao.js to retrieve user data and validate credentials. The workflow of the “src/db” directory consists of managing SQL queries interaction by managing the connection to SQLite inside database.js file, retrieving article links by id inside GetArticleLink.js, as well as handling likes and user profiles inside likes.js and userprofile.js. Express.js middleware for authentication and security implemented under “src/middleware” inside auth-middleware.js, where it verifies JWT tokens in cookies and retrieves user details from tokens and attaches them and returns “404” status when the authentication fails. It works by making a request to a protected API endpoint, and when it is valid, it allows the request. API routes and entry points inside routes.js are developed under “src/routes” and consist of single API endpoint files.



*Figure: API routes files*

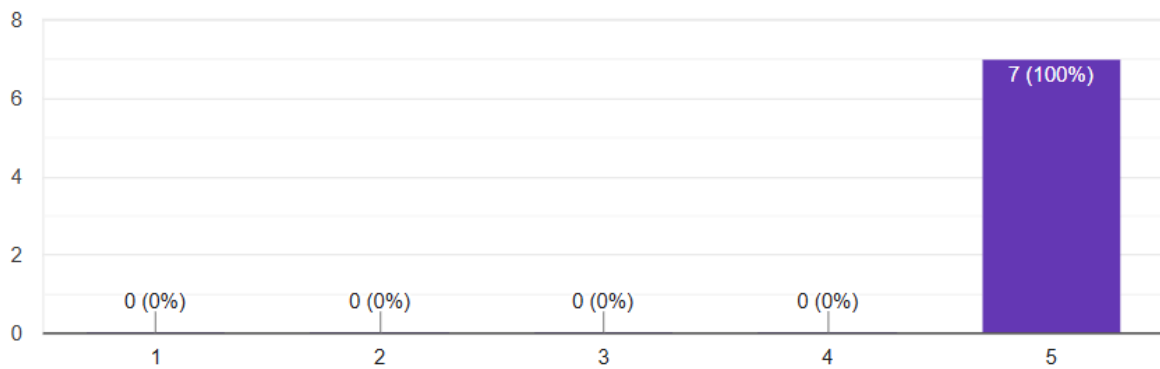
The Postman software comes in as the vital role of API testing via localhost port number. Under “src/sql”, the commands of various SQL schemas initiate the process of creating and inserting data into tables. “src/utlis” directory includes the purpose of generating and verifying JWT tokens to authenticate users securely and verifying a secret key with valid token as well as taking a username and setting a 24 hour expiry. The way of usage is when a user logs in, the server generates tokens and stores them in a cookie, where the token is required for accessing protected routes. Under the directory of “src” app.js is the main entry point of all backend features with Express.js server, loading middlewares, initiating the database connection, and setting up the server which includes all the functions listening on the specified port number.



The homepage is appealing.

 [Copy chart](#)

7 responses

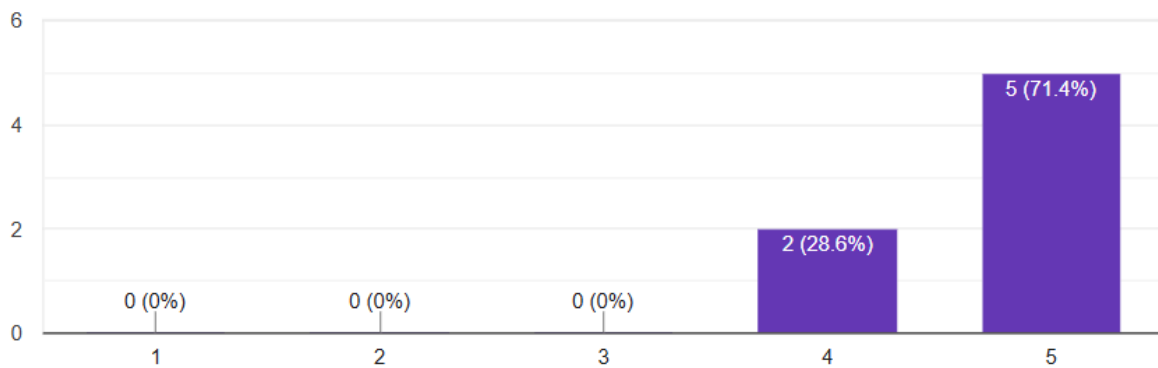


*Figure : User Testing Iteration 1 = "The homepage is appealing"*

The login page is appealing.

 [Copy chart](#)

7 responses

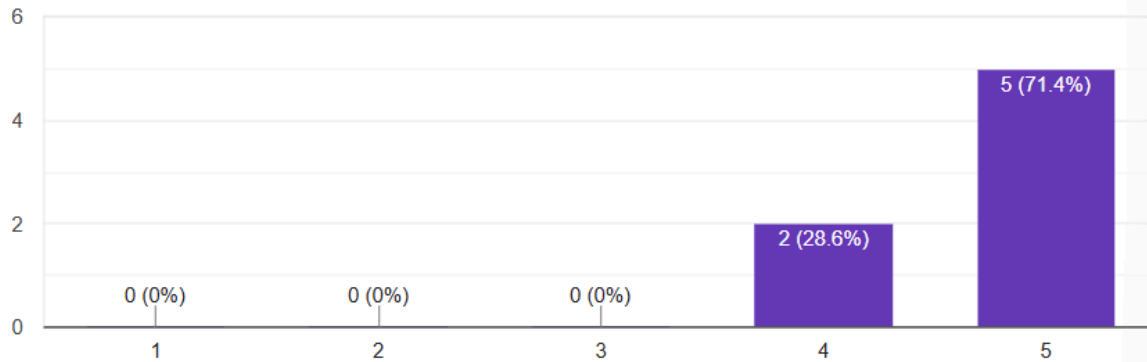


*Figure : User Testing Iteration 1 = "The login page is appealing"*

The Signup page is appealing.

 Copy chart

7 responses

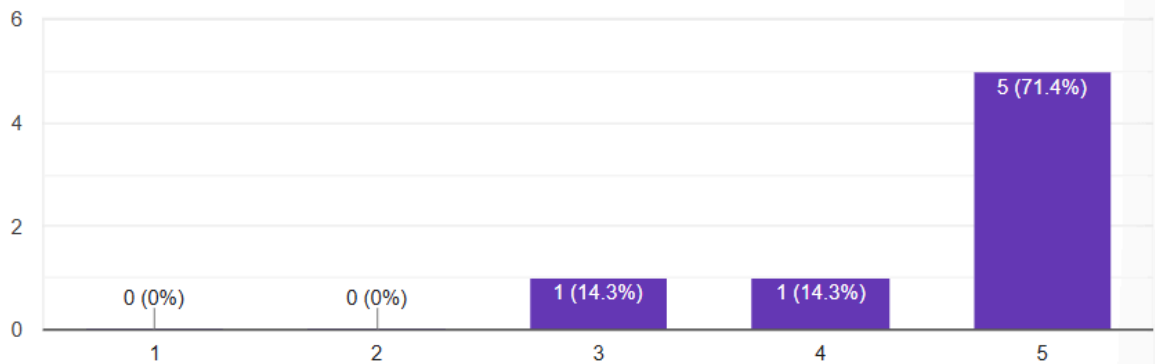


*Figure : User Testing Iteration 1 = “The Signup page is appealing”*

The Article Publishing page is appealing.

 Copy chart

7 responses

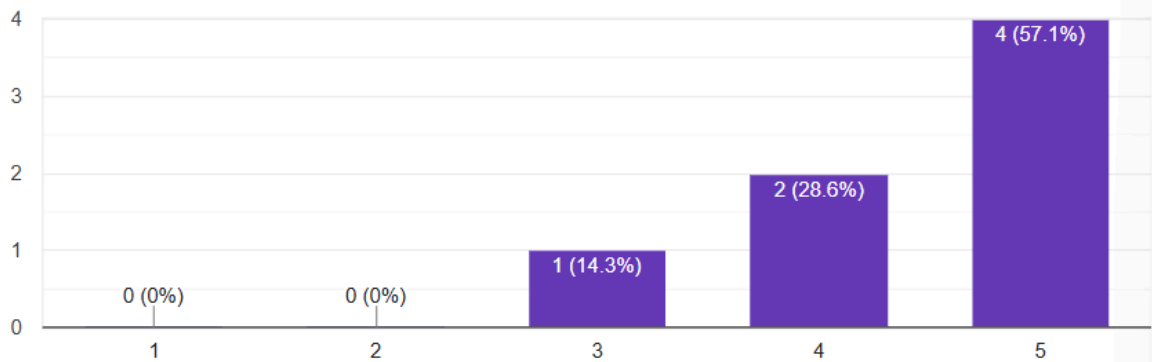


*Figure : User Testing Iteration 1 = “The Article Publishing page is appealing”*

The Individual Recipe page is appealing.

 Copy chart

7 responses

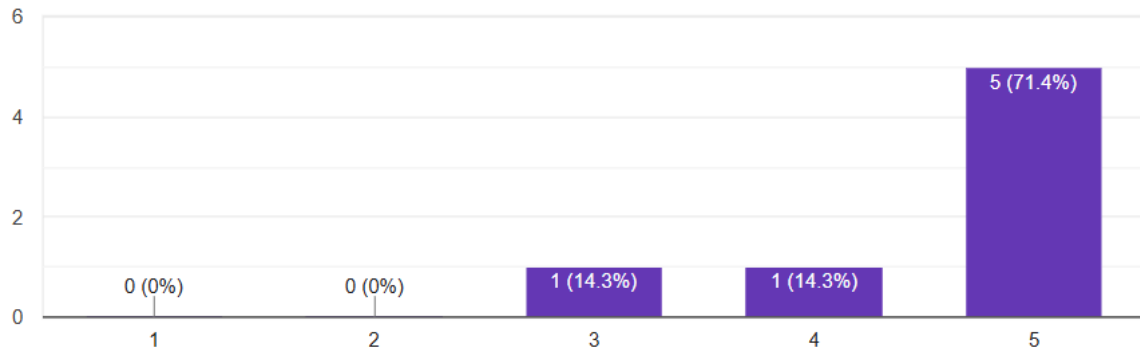


*Figure : User Testing Iteration 1 = “The Individual Recipe page is appealing”*

The recipe page is appealing

 [Copy chart](#)

7 responses



*Figure : User Testing Iteration 1 = “The Recipe page is appealing”*

Overall, the board testers thought that the various pages were good, with the majority choosing ‘Agree’ (4) or ‘Strongly Agree’ (5), making the average score to be 4.5. This data indicates that the changes made from the high fidelity to the new prototypes were successful in terms of design. Our testers rated 5 for the ease of logging into/creating an account, showing us that users are able to navigate the web page smoothly as they could locate the correct buttons and paths of action to take easily.

## 5. Analysis and Evaluation of Our Project (Soulful Spoon)

### 5.1 Introduction

#### 5.1.1 Problem Statement

The increasing prevalence of ultra-processed foods (UPFs) has been linked to various health concerns, including obesity, cardiovascular diseases, and diabetes (British Heart Foundation, n.d.). Despite the growing awareness of these issues, there is a lack of digital platforms that emphasize healthy eating while ensuring accessibility and inclusivity. Furthermore, young adults are shifting towards a health narrative that prioritizes mental well-being over calorie counting, a perspective that these platforms often fail to address.

As such, these individuals rely on social media platforms like TikTok and Instagram for healthy eating advice, filling the gap left by educators, the food industry, and policymakers (EIT Food, 2021).

#### 5.1.2 Existing Solutions and Their Limitations

While various recipe-sharing platforms and food blogs exist, they often fail to adequately address the following key concerns:

- **Limited Focus on Healthy Recipes:** Many platforms prioritize engagement metrics over nutritional value, making it challenging for users to find recipes that support their health goals.
- **Lack of Holistic Approach to Nutrition and Well-Being:** Many platforms predominantly focus on calorie counting as the primary strategy for health improvement. However, many dietitians argue that true health benefits stem from a balanced diet, nutrient-dense foods, and mindful eating practices, rather than simply reducing calorie intakes (Novak, 2025).
- **Limited Dietary-Specific Recipes:** Many platforms lack dietary-specific recipes (e.g., Vegan, Gluten-Free, Halal options), limiting accessibility for users with specific dietary needs.
- **Weak Community Engagement:** Many platforms rarely provide meaningful discussion spaces or credibility mechanisms for user-submitted recipes. This reduces trust and interaction among users.
- **Advertisement Overload:** Many platforms rely excessively on advertisements, disrupting user experience.
- **Lack of Personalisation and Accessibility:** Some platforms lack artificial intelligence (AI) chatbots for tailored recipe recommendations and guidance, dietary-specific adjustments, and accessibility features (e.g. dark/light mode, font size customization), reducing user engagement and inclusivity.

## 5.2 Soulful Spoon

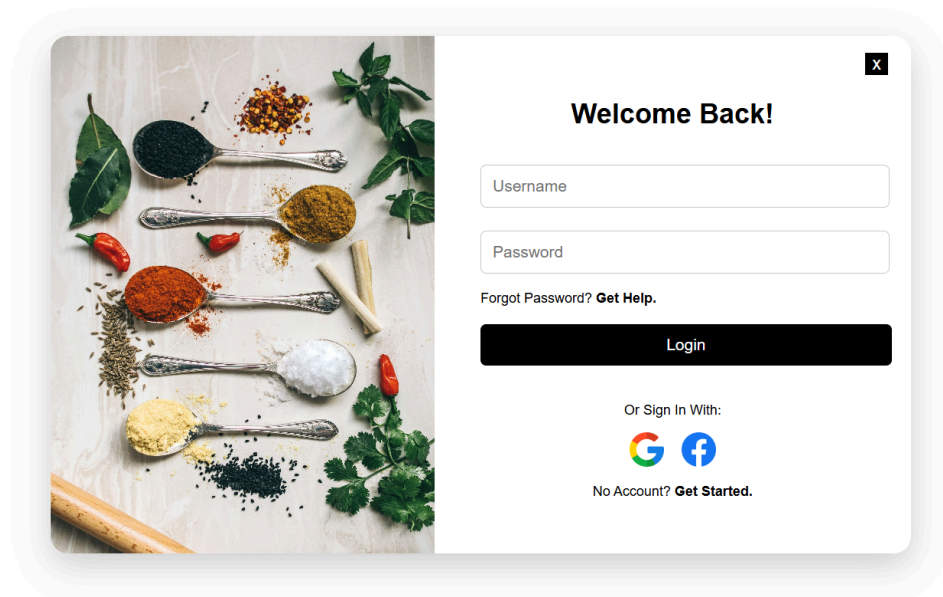
To address the aforementioned concerns, we developed **Soulful Spoon**, a health-focused recipe-sharing platform that offers an interactive and user-friendly experience, promoting nutritious eating and community engagement.

This section analyses the implemented solutions by assessing their effectiveness and exploring potential improvements. This analysis draws on literature, best practices, and industry standards for a rigorous evaluation.

### 5.2.1 Overview of Soulful Spoon

#### Login Page (/login)

- User authentication via username and password
- User authentication via Google through Firebase
- Link to Sign Up Page





```

8
9   let user = null;
10  let username = "";
11  let password = "";
12  let error = false;
13
14  async function googleLogin(){
15    try{
16      const result = await loginWithGoogle();
17      user = result.user;
18      console.log("Google Login Sucess:", user);
19      // goto("/", { replaceState: true });
20      window.location.href = "/"; //had to change to this to ensure the page reloads completely
21
22    } catch (error){
23      console.log("Google Login Failed:", error);
24    }
25  }
26
27  function handleLogout(){
28    logout();
29    user = null;
30    console.log("User is logged out");
31  }
32
33  onMount(() => {
34    auth.onAuthStateChanged((u) => {
35      if(u){

```

```

47    */
48    async function handleSubmit() {
49      error = false;
50
51      try {
52        const response = await fetch(AUTH_URL, {
53          method: "POST",
54          credentials: "include",
55          headers: { "Content-Type": "application/json" },
56          body: JSON.stringify({ username, password })
57        });
58
59
60        if (!response.ok) {
61          // If response is not okay (status code other than 2xx)
62
63          if (response.status === 401) {
64            // Unauthorized: Incorrect username or password
65            error = true;
66          } else {
67            // Other server errors
68            console.error("Server error:", response.statusText);
69          }
70        } else {
71          // Successful login, redirect user to homepage
72          goto("/", { invalidateAll: true, replaceState: true });
73        }
74      } catch (error) {

```

```

84 <!-- HTML for login page-->
85 <div class="login-container" >
86   <!-- login image -->
87   <div class="login-image"></div>
88   <!-- login form -->
89   <div class="login-form">
90     <!-- close button -->
91     <!-- <button class="close-button" on:click={() => goto('/')}>x</button> -->
92     <button class="close-button" on:click={() => window.location.href = "/"}>x</button>
93     <h2>Welcome Back!</h2>
94     <form on:submit|preventDefault={handleSubmit}>
95       <input type="username" bind:value={username} placeholder="Username" required />
96       <input type="password" bind:value={password} placeholder="Password" required />
97       <p class="forgot-password">
98         Forgot Password? <a href="#"> Get Help</a>
99       </p>
100       <button class="login-button" type="submit">Login</button>
101     </form>
102
103     {#if user}
104     <div class="user-info">
105       <img src={user.photoURL} alt="Profile">
106       <p>Welcome, {user.displayName}</p>
107       <button on:click={handleLogout}>Logout</button>
108     </div>
109     {:else}
110     <div class="social-login">
111       <p>Or Log In With:</p>

```

Figure 1: Login Page

## Sign Up Page (/signup)

- User registration
  - Username
  - Email
  - First Name
  - Last Name
  - Password
  - Confirm Password
  - Profile Picture (6 preset options and file upload)
- User authentication via Google through Firebase
- Link to Login Page



X

## Create Account

Username:

Email:

First Name:

Last Name:

Password:

Confirm Password:

Profile Picture:



Choose File

No file chosen

Next

Sign Up

Or Sign Up With:





Already Have an Account? [Login](#)

```
1 let currentImage = 0;
2 let selectedImage = images[currentImage];
3
4 async function googleLogin() {
5   try {
6     const result = await loginWithGoogle();
7     user = result.user;
8     const uniqueUsername = user.displayName.replace(" ", "_") + "_" + Math.floor(Math.random() * 10000);
9
10    const response = await fetch(SIGNUP_URL, {
11      method: "POST",
12      credentials: "include",
13      headers: { "Content-Type": "application/json" },
14      body: JSON.stringify({
15        username: uniqueUsername,
16        email: user.email,
17        profilePicture: user.photoURL
18      })
19    });
20
21    if (!response.ok) throw new Error("Google sign-up failed");
22
23    console.log("Google Login Success:", user);
24    window.location.href = "/";
25  } catch (err) {
26    console.error("Google Login Error:", err);
27    error = true;
28  }
29 }
```

```

61 async function signupUser() {
62   if (password !== confirmedPassword) {
63     passwordMatchError = true;
64     return;
65   }
66
67   try {
68     const response = await fetch(SIGNUP_URL, {
69       method: "POST",
70       credentials: "include",
71       headers: { "Content-Type": "application/json" },
72       body: JSON.stringify({
73         username,
74         firstName,
75         lastName,
76         email,
77         password,
78         profilePicture: selectedImage
79       })
80     });
81
82     if (response.status === 409) {
83       const errorData = await response.json();
84       if (errorData.message.includes("username")) usernameTakenError = true;
85       if (errorData.message.includes("email")) emailTakenError = true;
86       return;
87     }
88   }
89 }
90
91 function changeProfilePicture() {
92   currentImage = (currentImage + 1) % images.length;
93   selectedImage = images[currentImage];
94 }
95
96 </script>
97
98 <main>
99   <h1>Sign Up</h1>
100
101   <form on:submit|preventDefault={signupUser}>
102     <input type="text" bind:value={username} placeholder="Username" required />
103     {#if usernameTakenError}
104       <p class="error">Username is already taken.</p>
105     {/if}
106
107     <input type="text" bind:value={firstName} placeholder="First Name" required />
108     <input type="text" bind:value={lastName} placeholder="Last Name" required />
109
110     <input type="email" bind:value={email} placeholder="Email" required />
111     {#if emailTakenError}
112       <p class="error">Email is already registered.</p>
113     {/if}
114
115     <input type="password" bind:value={password} placeholder="Password" required />
116     <input type="password" bind:value={confirmedPassword} placeholder="Confirm Password" required />
117     {#if passwordMatchError}
118       <p class="error">Passwords do not match.</p>
119     {/if}
120   </form>
121 </main>

```

Figure 2: Sign Up Page

## Home Page (/)

- Dynamic recipe rendering
  - Recipes stored in the database are fetched and dynamically displayed on the homepage
  - Ensures users can see the latest published recipes in real time.
- Navigation Bar
  - Logo (Links to Home Page)
  - Name
  - Recipes (Links to Recipes Page)
  - Tips and Tricks (Links to Tips and Tricks Page)
  - About Us (Links to About Us Page)
  - "+" Button (Link to Publish Recipe Page)
  - Dark / Light Mode Toggle
  - Profile Picture
    - Login Option (Links to Login Page)
    - Sign Up Option (Links to Sign Up Page)
    - Sign Out
    - User Profile (Links to User Dashboard Page)
- Filtering and Search Functionalities
  - Course: Breakfast, Lunch, Dinner, Bites, Dessert, High Protein and Baking
  - Diets: Vegan, Vegetarian, Halal, Gluten-Free, and Dairy Free
- AI Chatbot using Botpress
- Main Content
  - 8 sample recipes
  - User-published recipes are dynamically fetched and displayed
- Footer
  - Quick Links to Recipes Page, Tips and Tricks Page and About Us Page
  - Socials (Instagram, Facebook, X, Whatsapp)

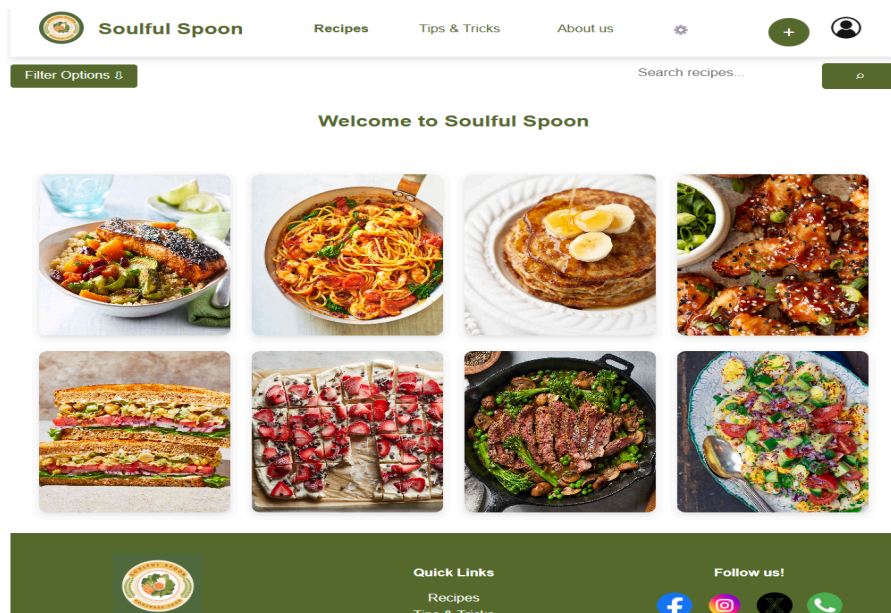


Figure 3: Home Page

## Recipe Page (/recipes)

- Navigation Bar
  - Logo (Links to Home Page)
  - Name
  - Recipes (Links to Recipes Page)
  - Tips and Tricks (Links to Tips and Tricks Page)
  - About Us (Links to About Us Page)
  - "+" Button (Link to Publish Recipe Page)
  - Profile Picture
    - Login Option (Links to Login Page)
    - Sign Up Option (Links to Sign Up Page)
    - Sign Out
    - User Profile (Links to User Dashboard Page)
- AI Chatbot using Botpress
- Main Content
  - 28 sample recipes categorised into structured sections
- Footer
  - Quick Links to Recipes Page, Tips and Tricks Page and About Us Page
  - Socials (Instagram, Facebook, X, Whatsapp)



## Breakfast Recipes

Start your day with the best breakfast options.



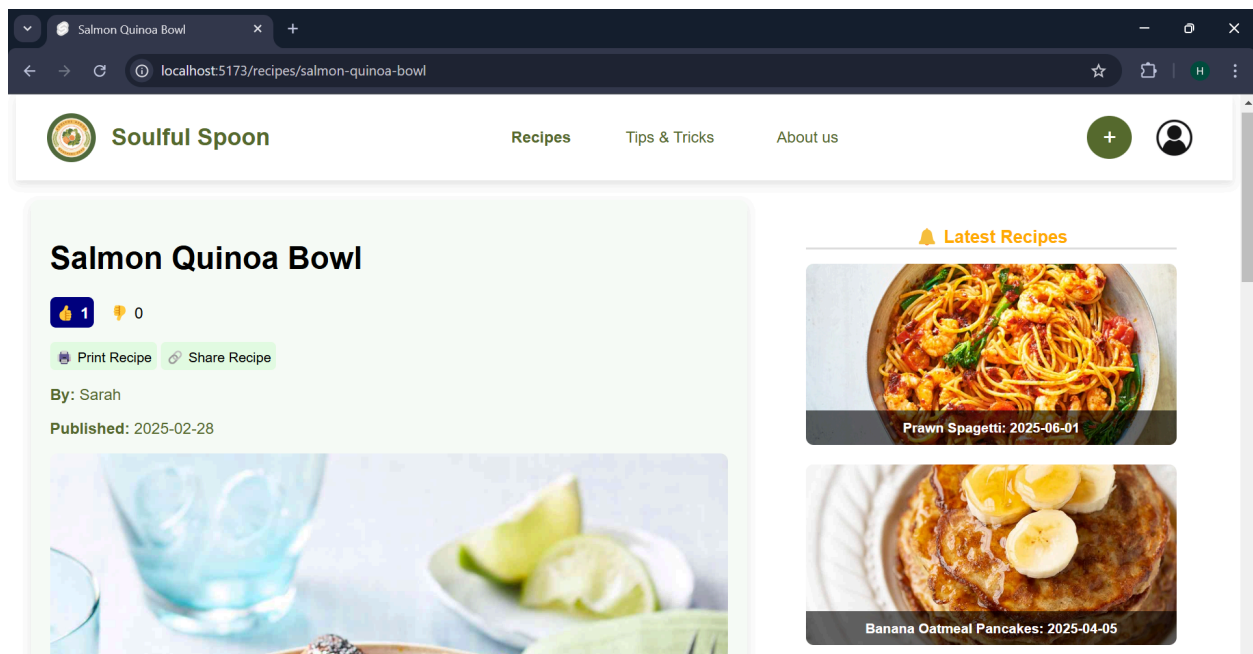
Figure 4: Recipe Page

### Individual Recipe Page (/recipes/[recipe-name])

- Individual Recipe Pages are dynamically rendered that adapts to the specific data of each recipe.
- Navigation Bar
  - Logo (Links to Home Page)
  - Name
  - Recipes (Links to Recipes Page)
  - Tips and Tricks (Links to Tips and Tricks Page)
  - About Us (Links to About Us Page)
  - "+" Button (Link to Publish Recipe Page)
  - Profile Picture
    - Login Option (Links to Login Page)
    - Sign Up Option (Links to Sign Up Page)
    - Sign Out
    - User Profile (Links to User Dashboard Page)
- Recipe Container
  - Title
  - Author
  - Published Date
  - Interactive Buttons



- Like and Dislike Buttons (1 user = 1 reaction, even if user is not authenticated)
  - Print Recipe Button (Only recipe content is printed, details such as navigation bar, likes, dislikes, comments and etc are hidden)
  - Share Recipe Button (Integration of Whatsapp, Facebook and Twitter APIs)
- Recipe Image
- Recipe Content
- Comments
  - Viewing (Accessible to all users)
  - Commenting & Replying (Restricted to authenticated users). Replying to a reply automatically includes a user tag
  - Deleting (Restricted to authenticated users who posted the comment). If a user deletes his comment, the thread of replies are also deleted.
- Dynamic Content Section
  - Latest Articles: Dynamically renders the top 3 most recently published recipes.
  - Most Liked: Dynamically renders the top 3 most liked recipes.
- Email Updates Subscription
  - Authenticated users have the option to subscribe and unsubscribe to our email updates
- Footer
  - Quick Links to Recipes Page, Tips and Tricks Page and About Us Page
  - Socials (Instagram, Facebook, X, Whatsapp)







## Recipe Details

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras fringilla tellus mauris. Integer congue eros ex, ut condimentum ipsum egestas quis. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. In interdum condimentum turpis, sit amet posuere ex pharetra in. Duis sollicitudin nibh lobortis fringilla pharetra. Vestibulum mollis ligula ut diam iaculis, vel pellentesque metus tristique. Donec egestas massa lacus, a accumsan lacus semper sed. Integer imperdiet turpis ac dapibus cursus. Donec ac molestie est.

## Comments

Post Comment



**TestUser**  
nice!

3/5/2025, 12:45:20 PM

Reply



**TestUser**  
great

3/5/2025, 12:45:25 PM

Reply



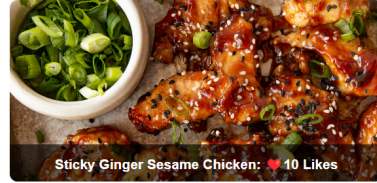
**TestUser**  
Hi testing

3/5/2025, 12:46:50 PM

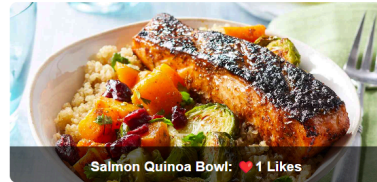
Reply

Sticky Ginger Sesame Chicken: 2025-03-30

## Most Liked Recipes



Sticky Ginger Sesame Chicken: 10 Likes



Salmon Quinoa Bowl: 1 Likes



Prawn Spagetti: 1 Likes



## Stay Updated

Subscribe to receive new recipes and cooking tips!

☐

I want to recieve updates at

```

59     }
60
61     function toggleShareOptions(event){
62         event.stopPropagation();
63         showShareMenu = !showShareMenu;
64     }
65
66     function handleClickOutside(event){
67         if(!event.target.closest(".share-container")){
68             showShareMenu = false;
69         }
70     }
71
72     function shareOn(platform){
73         let text = encodeURIComponent("Check Out This Recipe: ");
74         let url = encodeURIComponent(currentURL);
75         let shareURL = "";
76
77         if (platform === "whatsapp"){
78             shareURL = `https://api.whatsapp.com/send?text=${text}%20${url}`;
79         }
80         else if (platform === "facebook"){
81             shareURL = `https://www.facebook.com/sharer/sharer.php?u=${url}`;
82         }
83         else if (platform === "twitter"){
84             shareURL = `https://twitter.com/intent/tweet?text=${text}&url=${url}`;
85         }

```

```

    }

    function addComment(){
        if (newComment.trim() === ""){
            return;
        };

        const commentObject= {
            id: Date.now(),
            username: loggedInUser,
            text: newComment,
            date: new Date().toLocaleString(),
            profilePic: profileicon,
            replies: []
        };

        comments = [...comments, commentObject];
        localStorage.setItem(`comments-${id}`, JSON.stringify(comments));
        newComment = "";
    }

    function deleteComment(commentID){
        comments = comments.filter(comment => comment.id !== commentID);
        localStorage.setItem(`comments-${id}`, JSON.stringify(comments));
    }

    function addReply(commentID){
        if(replyText.trim() === ""){

```

```

251 function deleteReply(commentID, replyID){
252     comments = comments.map(comment => {
253         if (comment.id === commentID){
254             return{
255                 ...comment,
256                 replies: comment.replies.filter(reply => reply.id !== replyID)
257             };
258         }
259         return comment;
260     });
261     localStorage.setItem(`comments-${id}`, JSON.stringify(comments));
262 }
263
264 function toggleComments(){
265     commentsExpanded = !commentsExpanded;
266     visibleComments = commentsExpanded ? comments.length : 7;
267 }
268
269 function toggleReplies(commentID){
270     visibileReplies[commentID] = !visibileReplies[commentID];
271 }
272
273 function toggleSubscription(event){
274     if(!loggedInUserEmail) return;
275
276     isSubscribed = event.target.checked;
277     localStorage.setItem(`newsletter-${loggedInUserEmail}`, JSON.stringify(isSubscribed));
278 }

```

```

JS +page.js x
Client-Side > src > routes > recipes > [slug] > JS +page.js
1  export async function load ({params, fetch}){
2      try{
3          const response = await fetch('/recipes.json');
4
5          if(!response.ok){
6              throw new Error('Failed to fetch recipes.json: ${response.status}');
7          }
8
9          const recipes = await response.json();
10
11         const recipe = recipes.find(r => r.id.toLowerCase().trim() === params.slug.toLowerCase().trim());
12
13         if(!recipe){
14             return {status: 404, error: new Error("Recipe not found")};
15         }
16         return {recipe};
17     }
18
19     catch (error){
20         console.error("Error loading recipes:", error.message);
21         return {status: 500, error: new Error("Failed to load recipe")};
22     }
23
24 }

```

```
+page.svelte X
Client-Side > src > routes > recipes > [slug] > +page.svelte > ...
Legacy mode
1 <script>
2   import ArticleDetail from "$lib/components/Articles/ArticleDetail.svelte";
3   export let data;
4 </script>
5
6 <svelte:head>
7   <title>{data.recipe.article_title}</title>
8   <script src="https://cdn.botpress.cloud/webchat/v2.2/inject.js">
9 </script><script src="https://files.bpcontent.cloud/2025/02/25/06/20250225063934-HGA74MAU.js"></script>
10 </svelte:head>
11
12 {#if data?.recipe}
13 <ArticleDetail
14   id={data.recipe.id}
15   article_title={data.recipe.article_title}
16   article_content={data.recipe.article_content}
17   username={data.recipe.username}
18   likes={data.recipe.likes}
19   dislikes={data.recipe.dislikes}
20   date_published={data.recipe.date_published}
21   image_path={data.recipe.image_path}
22 />
23 {:else}
24   <p>Loading Recipe...</p>
25 {/if}
```

Figure 5: Individual Recipe Page

## Recipe Publish Page (/mockarticlepublish)

- Only authenticated users are able to publish a recipe. If a user is not authenticated and goes to the recipe publish page, it will prompt the user to login.
- Navigation Bar
  - Logo (Links to Home Page)
  - Name
  - Recipes (Links to Recipes Page)
  - Tips and Tricks (Links to Tips and Tricks Page)
  - About Us (Links to About Us Page)
  - "+" Button (Link to Publish Recipe Page)
  - Profile Picture
    - Login Option (Links to Login Page)
    - Sign Up Option (Links to Sign Up Page)
    - Sign Out
    - User Profile (Links to User Dashboard Page)
- Recipe Publish Container
  - Recipe Title
  - Course Selection: Breakfast, Lunch, Dinner, Bites, Dessert, High Protein and Baking (User can select multiple)
  - Diet Selection: Vegan, Vegetarian, Halal, Gluten-Free, and Dairy Free (User can select multiple)
  - Recipe Details (TinyMCE Editor)
  - Image Upload
  - Publish Recipe Button
- Footer
  - Quick Links to Recipes Page, Tips and Tricks Page and About Us Page
  - Socials (Instagram, Facebook, X, Whatsapp)



## Publish Your Recipe!

By: **Tester**

**Title**

**Course**

Breakfast Lunch Dinner Snack Dessert

**Diet**

Vegan Halal Gluten-Free Keto Vegetarian

**Recipe Content**

File Edit View Insert Format

↶ ↷ Paragraph

**B** *I*

...

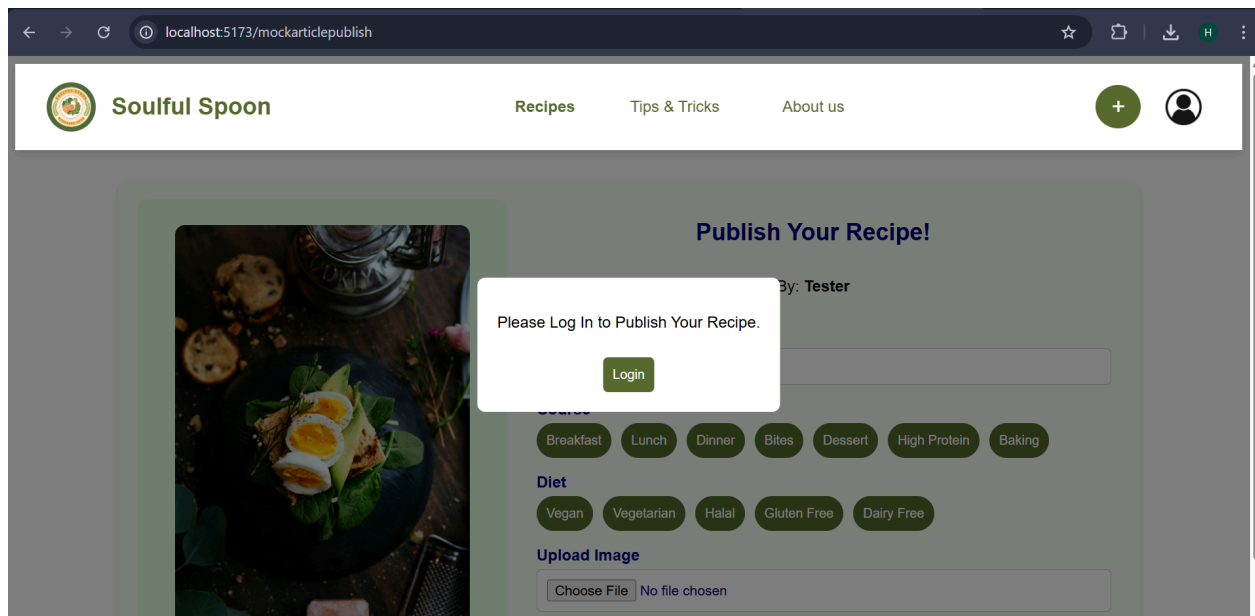
p

Build with  tinyMCE

**Upload Image**

Choose File No file chosen

Publish Recipe



```

31
32   async function createArticle(event) {
33       event.preventDefault();
34
35       if (!user) {
36           error = "You must be logged in to publish an article.";
37           return;
38       }
39
40       let newArticle = {
41           title: article_title,
42           content: article_content,
43           username,
44           likes,
45           dislikes,
46           date_published,
47           image_path,
48           course: selectedCourse,
49           diet: selectedDiet
50       };
51
52       try {
53           const response = await fetch(POST_URL, {
54               method: "POST",
55               credentials: "include",
56               headers: { "Content-Type": "application/json" },
57               body: JSON.stringify(newArticle)
58           });
59

```

```

75
76   function handleFileChange(event) {
77       const file = event.target.files[0];
78       if (file) {
79           const reader = new FileReader();
80           reader.onload = function(e) {
81               image_path = e.target.result;
82           };
83           reader.readAsDataURL(file);
84       }
85   }
86
87   function toggleCourse(course) {
88       if (selectedCourse.includes(course)) {
89           selectedCourse = selectedCourse.filter(c => c !== course);
90       } else {
91           selectedCourse = [...selectedCourse, course];
92       }
93   }
94
95   function toggleDiet(diet) {
96       if (selectedDiet.includes(diet)) {
97           selectedDiet = selectedDiet.filter(d => d !== diet);
98       } else {
99           selectedDiet = [...selectedDiet, diet];
100       }
101   }
102 </script>
103

```

```

139 <div class="article-page {showLoginPopup ? 'blur-background' : ''}">
140   <div class="publish-container">
141     <div class="image-container">
142       <img src={image_path} alt="Recipe Image">
143     </div>
144     <div class="form-container">
145       <h2>Publish Your Article</h2>
146       <p class="username">By: <strong>{username}</strong></p>
147       <form on:submit|preventDefault={createArticle}>
148         <label>Title</label>
149         <input type="text" bind:value={article_title} placeholder="Enter Title" required />
150
151         <label>Course Category</label>
152         <div class="course-selection">
153           {#each courseOptions as course}
154             <button type="button" class="course-button {selectedCourse.includes(course) ? 'selected'
155               {course}
156             </button>
157           {/each}
158         </div>
159
160         <label>Dietary Preferences</label>
161         <div class="diet-selection">
162           {#each dietOptions as diet}
163             <button type="button" class="diet-button {selectedDiet.includes(diet) ? 'selected' : ''}
164               {diet}
165             </button>
166           {/each}

```

Figure 6: Publish Recipe Page

## Tips & Tricks (/Tips&tricks)

- Navigation Bar
  - Logo (Links to Home Page)
  - Name
  - Recipes (Links to Recipes Page)
  - Tips and Tricks (Links to Tips and Tricks Page)
  - About Us (Links to About Us Page)
  - "+" Button (Link to Publish Recipe Page)
  - Dark / Light Mode Toggle
  - Profile Picture
    - Login Option (Links to Login Page)
    - Sign Up Option (Links to Sign Up Page)
    - Sign Out
    - User Profile (Links to User Dashboard Page)
- Kitchen Hacks & Cooking Tips:
  - Section containing various cooking hacks and kitchen organization tips
  - Read more button to allow users to expand details about tools and techniques.
- Embedded Videos for Learning:
  - Section containing various cooking hacks and kitchen organization tips
- Section containing various cooking hacks and kitchen organization tips:
  - Cooking tutorials and kitchen efficiency tips embedded via YouTube.
- Dynamic Content Display:



- Information is structured with an expandable section to avoid overwhelming users. Users can toggle visibility for detailed explanations of kitchen tools and tricks.
- Health & Nutrition Insights:
  - Highlights benefits of using kitchen tools like an air fryer. Focus on healthier eating methods.
  - Offers guidance on reducing unhealthy fats and improving nutrition in everyday meals.
- Footer
  - Quick Links to Recipes Page, Tips and Tricks Page and About Us Page
  - Socials (Instagram, Facebook, X, Whatsapp)

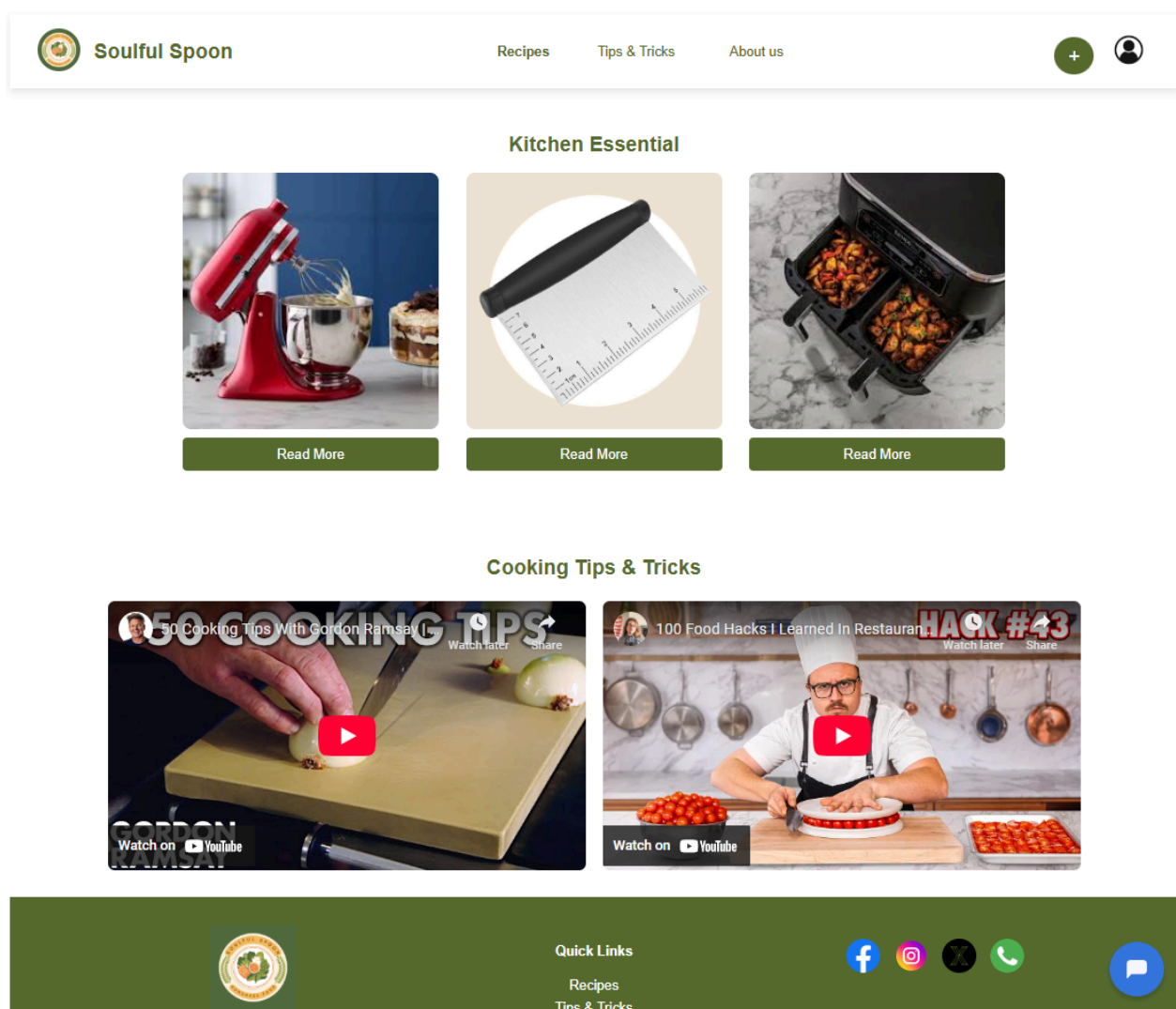


Figure 7: Tips and Tricks Page

## About us (/Aboutus)

- Navigation Bar
  - Logo (Links to Home Page)
  - Name
  - Recipes (Links to Recipes Page)
  - Tips and Tricks (Links to Tips and Tricks Page)
  - About Us (Links to About Us Page)
  - "+" Button (Link to Publish Recipe Page)
  - Dark / Light Mode Toggle
  - Profile Picture
    - Login Option (Links to Login Page)
    - Sign Up Option (Links to Sign Up Page)
    - Sign Out
    - User Profile (Links to User Dashboard Page)
- Overview:
  - Provides an introduction to Soulful Spoon and its mission.
  - Highlights the platform's goal of promoting healthy eating and inclusive dietary choices.
  - Offers a community-driven approach, where users can share and discover recipes.
- Contact & Feedback System:
  - Users can reach out via email.
  - A contact form allows users to send messages directly to the platform.
  - Stored in database: All user-submitted messages are securely stored in the database for future reference and support purposes.
- Footer
  - Quick Links to Recipes Page, Tips and Tricks Page and About Us Page
  - Socials (Instagram, Facebook, X, Whatsapp)

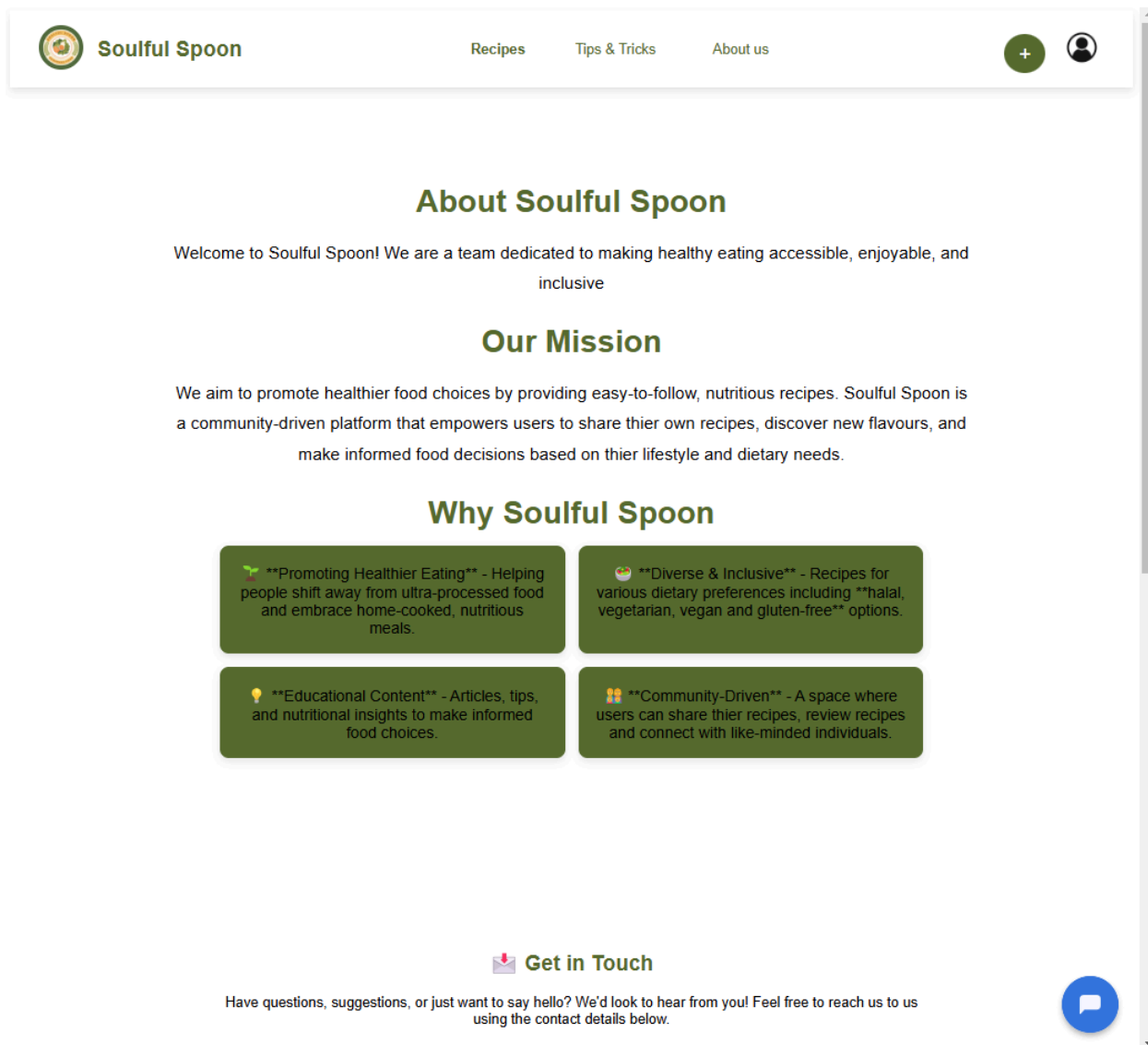


Figure 8: About Us Page

## User Dashboard (/Userdashboard)

- Centralized profile management for registered users.
- Profile management:
  - Provides options for updating and editing personal details.
  - Users can choose the font-size, dark mode/light mode.



 **User Dashboard**

**Username**

**Bio**

**Email / Phone**

Save Profile

 **Preferences**

**Website Color**  

Light Mode

**Text Size**  

Medium

Save Preferences



Figure 8: User Dashboard Page

### 5.2.2 Building Soulful Spoon: Technology Stack and Implementation (Analysis + Evaluation)

**Frontend:** The frontend was designed to be interactive and lightweight, providing a smooth user experience.

- **Svelte:** Svelte was chosen as the frontend framework due to its reactive architecture and minimal runtime overhead. Unlike popular frameworks like React and Vue, Svelte compiles components at build time, resulting in (Bhushan, 2023):
  - Faster speed and performance due to smaller bundler sizes.
  - Optimized reactivity that eliminates the need for a virtual DOM.
  - Simplified state management, minimising complexity.
- **SvelteKit:** SvelteKit was chosen as the meta-framework for Svelte for routing, state management and server-side rendering (SSR) (Svelte, n.d.). Furthermore, its build optimization compiles code into efficient JavaScript and CSS for faster load times (Sanity, 2024). It also supports offline functionality through Progressive Web Apps (PWAs). Additionally, Hot Module Replacement (HMR) enables real-time code updates, improving development speed.
- **Vanilla CSS:** Initially, Tailwind CSS was considered, but due to compatibility challenges with the Svelte framework, we opted for Vanilla CSS instead.

**Backend:** The backend was designed to be lightweight, scalable, and efficient in handling data storage and user interactions.

- **SQLite3:** SQLite3 was chosen as the database for Soulful Spoon. We used it to store user data, recipes, likes, dislikes, comments, etc. It was ideal for the following reasons:
  - Reliability and Performance: Ensures smooth recipe storage and retrieval.
  - Easy Setup and Maintenance: Requires minimal configuration.
  - Lightweight: Ideal for medium-scale applications.
- **Node.js:** Node.js was used as the runtime environment to support asynchronous operations, scalability and integration with Express.js.
- **Express.js:** Express.js was used for handling HTTP requests, managing middleware and creating RESTful endpoints for recipe retrieval and user authentication.
- **RESTful API:** RESTful API was used for:
  - Data communication between frontend and backend.
  - Efficient execution of CRUD (Create, Read, Update, Delete) operations for recipes and user interactions.
  - Modular design which allows for scalability.
- **Bcrypt:** Bcrypt was used for password hashing to enhance security.

## 5.3 Core Features in Soulful Spoon (Analysis + Evaluation)

### 5.3.1 Authentication

In Soulful Spoon, JWTs (JSON Web Tokens) are primarily for backend API security and user validation. This ensures that only authenticated users can access protected API routes. We also utilized SvelteKit's built-in features to manage frontend session persistence and user authentication. Our implementation follows industry best practices to enhance security and maintain efficiency.

#### 1. Secure Token Storage

★ **Best Practice:** JWTs should be stored securely to prevent attacks like Cross-Site Scripting (XSS) attacks (Trojanowski, 2024).

- We store JWTs in HTTP-only cookies (authToken), preventing access via JavaScript and mitigating XSS risks.

★ **Enhancement:** Enforcing SameSite and Secure flags would further protect against Cross-Site Request Forgery (CSRF) attacks (Endgrate Team, 2024).

#### 2. Token Expiration

★ **Best Practice:** Tokens should have short expiration times to limit the risk of misuse in case of compromise. Using refresh tokens enhances security and user experience (Trojanowski, 2024).

- We ensure JWTs expire after 24 hours, ensuring limited validity.
- Logout removes the cookie by setting an expiration date in the past.

★ **Enhancements:**

- Implementing refresh tokens could allow persistent sessions without requiring frequent login.
- Currently, logout does not revoke the token on the server. Therefore, an attacker with the token could still use it until expiration. Hence, implementing real-time revocation mechanisms would enhance security.

#### 3. Token Verification

★ **Best Practice:** Tokens must be verified (Trojanowski, 2024).

- The middleware extracts the 'authToken' and validates it using 'getUsernameFromJWT'. If valid, the user is granted access to protected API routes.

★ **Enhancements:**

- Currently, all authenticated users have the same level of access. Implementing Role-Based Access Control (RBAC) will allow us to define user roles and permissions. This ensures that users can only access routes appropriate to their authorisation level.
- Currently, JWT verification runs on every request, which could slow down the backend. To reduce redundant verification, cache decoded tokens in memory using a lightweight caching solution.

#### 4. Session Management

- ★ **Best Practice:** Implementing robust session management is crucial for maintaining secure and efficient user authentication (Endgrate Team, 2024).
  - We leveraged SvelteKit's capabilities to track user sessions, ensuring that authenticated users maintain access without frequent re-authentication.

#### 5. State Management

- ★ **Best Practice:** Efficient state management ensures that authentication status is consistently reflected across the application.
  - We used SvelteKit's writable stores to maintain and reactively update user authentication status through the application.

### 5.3.2 Google Authentication through Firebase

In Soulful Spoon, we allow users to authenticate using their Google accounts through Firebase, reducing the need for traditional username-password authentication while enhancing security. Our implementation aligns with industry best practices to ensure authentication efficiency and user convenience (GeeksforGeeks, 2024).

#### 1. Secure Authentication Handling

- ★ **Best Practice:** Authentication processes should be handled securely, leveraging industry-standard protocols like OAuth 2.0.
  - Users are able to authenticate using their Google accounts without storing sensitive credentials in our system.
  - Firebase handles OAuth 2.0 authentication flow, ensuring secure token exchange between client and the authentication provider (Firebase, 2020).
- ★ **Enhancement:**
  - Implement multi-factor authentication (MFA) for additional security measures (Endgrate Team, 2024).

#### 2. Session Persistence & Token Management

- ★ **Best Practice:** Secure token management is crucial to maintaining session integrity and preventing unauthorized access.
  - Firebase generates ID tokens to manage user sessions (Firebase, 2025).
  - Users remain logged in across sessions unless they actively logout.
  - Tokens are refreshed automatically to ensure uninterrupted access (Firebase, 2025).
- ★ **Enhancement:**
  - Enforce token invalidation upon logout to prevent unauthorized reuse.

#### 3. Protecting User Data & Privacy

- ★ **Best Practice:** Keep authentication data safe from threats like session hijacking, phishing and unauthorized access.

- Firebase encrypts user data and follows Google's security standards (Firebase, 2025).
- Sessions are secured with Firebase Security Rules to block unauthorized access.

★ **Enhancement:**

- Add reCAPTCHA to stop bot logins.

### 5.3.3 Course and Diet Based Filtering + Search Functionality

We have implemented course and diet-based filtering alongside a search functionality to enhance user experience.

1. Course and Diet-Based Filtering:

★ **Best Practice:** Implementing content filtering allows users to narrow down recipes based on specific requirements. This improves accessibility and relevance (DriveLock, 2024).

- Users can filter recipes by meal courses (Breakfast, Lunch, Dinner, Bites, Dessert, High Protein and Baking) and dietary preferences (Vegan, Vegetarian, Halal, Gluten-Free, and Dairy Free). Multiple-select filters are supported for more precise results.

★ **Enhancement:**

- Implement advanced algorithms or AI that learn user preferences over time to suggest personalized recipes.

2. Search Functionality:

★ **Best Practice:** Search functionality should be responsive and capable of handling various user inputs to provide precise results (Dunbar, 2024).

- The search bar is placed at the top of the page, making it easily accessible to users.

★ **Enhancements:**

- Expedite searches by providing autocomplete options and suggestions for popular searches.



- To ensure comprehensive results, the search functionality should account for common misspellings and recognize synonyms.

#### 5.3.4 AI Chatbot for Assistance (Botpress)

##### **Benefits of AI chatbots (Holdsworth, 2024):**

- **24/7 Availability:** Users can access information and support outside business hours.
- **Instant Responses:** Provides quick responses, reducing wait times, and enhancing satisfaction.
- **Cost Efficiency:** Reduces operational cost by handling routine inquiries, allowing contact personnel to focus on more complex tasks.

**Our Implementation:** Leveraging these advantages, our Botpress AI chatbot serves as a basic automated assistant. It answers general questions. This aligns with our commitment to utilizing technology to improve user engagement and satisfaction.

##### **Enhancements:**

- **Tailored Experience:** Train the chatbot on Soulful Spoon's content, enabling it to answer questions about our recipes and website features.
- **Smart Navigation:** Introduce dynamic redirection, guiding users to relevant pages based on their queries.
- **FAQ Section:** Add a dedicated FAQ section to help users find answers quickly and reduce repetitive interactions.

#### 5.3.5 User Engagement

User engagement is pivotal for fostering a vibrant community and ensuring the sustained success of Soulful Spoon.

For example, Reddit encouraged user-generated content (UGC) through subreddits, implemented an upvote and downvote system to ensure quality and created a discussion feature to keep users engaged. This resulted in high user retention and increased daily interactions (Reuters, 2025).

As such, implementing the features below will significantly enrich a user's experience.

1. **Like and Dislike Button:** Users can express their approval/disapproval of content, providing immediate feedback. The likes data is used to highlight popular recipes and improve recommendations.
  - Enhancements:
    - i. Create a "Liked Recipes" section in the User Dashboard for users to easily access their favourite recipes.
    - ii. Notify users when their recipe has been liked/disliked.

2. **Share Button:** Users can share recipes on platforms like WhatsApp, Facebook, and Twitter to increase reach and engagement.
  - Enhancement:
    - i. Enable sharing on other platforms like Telegram, Instagram, Pinterest, Reddit, etc.
3. **Comments:** Authenticated users can share their thoughts and engage in discussions on recipes. This allows for community building and active participation.
  - Enhancements:
    - i. Implement a robust moderation system to maintain a respectful environment.
    - ii. Notify users when their comment receives a reply to foster ongoing conversations.
4. **Email Updates Subscription:** Authenticated users can subscribe to receive the latest recipes, tips and tricks, and news.
  - Enhancement:
    - i. Tailor these updates based on user preferences and interaction history.
5. **Message (Feedback Form):** Users can easily share their experiences and suggestions.
  - Enhancement:
    - i. Send an automated reply acknowledging the feedback outside business hours.
6. **Publish Recipes (Authenticated Users):** Authenticated users can publish their own recipes using TinyMCE for rich text formatting. The published article will be dynamically rendered in the Home Page.
  - Enhancements:
    - i. Allow users to upload multiple recipe images.
    - ii. Allow users to create their own course and dietary options.
    - iii. Allow users to disable the like / dislike count.
    - iv. Allow users to disable comments.
    - v. Add a live preview mode to let users see their articles before publishing.

#### 5.3.6 Accessibility

Accessible websites are shown to achieve 35% better search results (Taiwo, 2024). Therefore, we have integrated the following adjustable features:

1. **Font Size Customization:** Users can increase or decrease font size for better readability.
2. **Dark / Light Mode Toggle:** Users can switch themes based on preference.

#### **Enhancements:**

- **Mobile Responsiveness:** Ensures website adapts to different screen sizes for seamless navigation
- **Save User Preferences:** Remember selected settings for future visits.
- **High Contrast Mode:** Improves visibility for users with low vision.

## 5.4 Areas for Improvement and Future Work

As Soulful Spoon continues to grow, several enhancements and new features can be introduced to further improve security, user experience and engagement. The following table highlights potential improvements across different pages. These upgrades will ensure Soulful Spoon emerges as a leading recipe-sharing and culinary inspiration hub.

| Page                   | Potential Improvements                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Login Page             | <ul style="list-style-type: none"><li>• Role-based access (i.e, users, admins)</li><li>• Multi-factor authentication for extra security</li><li>• Password reset</li><li>• Login with Facebook / X (Twitter)</li></ul>                                                                                                                                                                                                                                     |
| Sign Up Page           | <ul style="list-style-type: none"><li>• Email verification for manual sign up</li><li>• Sign up with Facebook / X (Twitter)</li></ul>                                                                                                                                                                                                                                                                                                                      |
| Home Page              | <ul style="list-style-type: none"><li>• More options for course and diet-based filtering</li><li>• More filters such as cooking time, difficulty level, and ingredients</li><li>• "For You" section with recipes tailored to each user through AI</li><li>• "Trending Recipes" section with recipes that have more engagement</li><li>• "Budget Bites" section with recipes that are cost-effective</li><li>• Streak / Gamification: Top authors</li></ul> |
| Individual Recipe Page | <ul style="list-style-type: none"><li>• Bookmark button for users, ratings, cooking time, and ingredients list</li><li>• Smart grocery list generator</li><li>• Nutritional data APIs to retrieve recipes and nutritional information</li><li>• Nutritional calculator</li></ul>                                                                                                                                                                           |
| Article Publish Page   | <ul style="list-style-type: none"><li>• More sections such as recipe description, cooking time, ingredients</li></ul>                                                                                                                                                                                                                                                                                                                                      |
| Tips and Tricks Page   | <ul style="list-style-type: none"><li>• User can publish news, tips, and tricks, and other cooking-relevant content</li></ul>                                                                                                                                                                                                                                                                                                                              |
| User Dashboard         | <ul style="list-style-type: none"><li>• Liked, saved, and published recipes</li><li>• Enable notifications for new recipes and content</li><li>• Badge if top 10 contributors</li></ul>                                                                                                                                                                                                                                                                    |
| Achievements Page      | <ul style="list-style-type: none"><li>• Leaderboards for top contributors</li></ul>                                                                                                                                                                                                                                                                                                                                                                        |

## 6. Evaluation

Team collaboration:

Version Control: Git and GitHub were used for tracking progress, managing branches, and resolving conflicts.

Project management: Google Docs and Gantt chart were used to define weekly tasks and milestones.

Testing and debugging: Debugging tools and browser dev tools were used to refine UI/UX

Strengths:

- Agile iterations for continuous refinements
- Effective use of Git/Github enables seamless collaboration
- Regular testing ensured early bug detection and resolution.

Challenges:

- More structured testing: Manual testing was performed, automated tests for key features such as login or filtering could improve reliability.

Key Learning:

- Iterative development process allowed for continuous improvements
- User engagement features (likes, dislikes, publishing) enhanced platform interactivity
- Implementing real-time filtering, chatbot api, authentication, and UI optimization improved functionality.

## 7. Conclusion/Summary

Through multiple iterations, we delivered a dynamic, community-driven recipe-sharing platform that integrates user-generated content, interactive features, and a seamless user experience. Our project achieved a functional platform that allows users to browse, filter, search, and interact with images. Personalised user experience through authentication and enables users to create content. Interactive engagement features such as article publishing, commenting, likes, and messaging, fostering a sense of community within the platform. A structured iterative development process, ensuring the system refinements based on feedback surveys. A responsive UI/UX with intuitive navigation and accessible interface.

## 8. Individual Report

My role:

During the development of Soulful Spoon, I contributed primarily to the frontend development and project planning and coordination with Harshikaa, and a small portion of the backend relating to my page. I ensured that the platform's design, **filtering system**, **AI chatbot**, messages, interactive features were implemented effectively. Developing the UI with Svelte (new framework) and CSS, to ensure accessibility and **seamless navigation**, **recipe filter** and **search system** using radio-button filtering for **course type and the dietary preferences**, and **footer**. Conducting manual testing and bug fixes, ensuring smooth navigation from each page to different pages. Made sure all pages were linked in a structured way.



#### Team Function:

Teamwork dynamics had its strengths and challenges. We adopted an agile development approach allowing continuous improvements. Effective team collaboration was important to the success of Soulful Spoon, but the workload was not distributed evenly, which led to delays and additional workload, causing Harshikaa and me to play a significant part in UI.

Throughout the project, we faced challenges with task completion as other team members required continuous follow-ups and frequent reminders.

Despite all these challenges, we managed to pull through as a team, ensuring essential features were implemented and completed. We also efficiently used Github for smooth version control, collaboration, and code management.

#### References

*Benefits of Chatbots*. (2024, January 18). IBM. Retrieved March 10, 2025, from

<https://www.ibm.com/think/insights/unlocking-the-power-of-chatbots-key-benefits-for-businesses-and-customers>

Bhushan, A. (2023, February 24). *Svelte vs React: Features, Performance, and More*. Kinsta.

Retrieved March 10, 2025, from <https://kinsta.com/blog/svelte-vs-react/>

*BMJ*. (2024). Ultra-processed food exposure and adverse health outcomes: umbrella review of epidemiological meta-analyses. <https://www.bmj.com/content/384/bmj-2023-077310>

*Engrate*. (2024, September 17). 10 Session Management Security Best Practices.

<https://endgrate.com/blog/10-session-management-security-best-practices>

*Enhancing On-Site Search Functionality: Best Practices for Websites*. (2024, May 21). Concrete

CMS. Retrieved March 10, 2025, from

<https://www.concretectms.com/about/blog/web-design/enhancing-on-site-search-functionality-best-practices-for-websites>

Ganz, S., & Figel, C. (2021, December 14). *How Some Viral TikTok Recipes Lead to Ingredient Shortages*. Eater. Retrieved March 10, 2025, from

<https://www.eater.com/22832651/viral-tiktok-recipes-food-shortages>

GeeksforGeeks. (2024, May 10). What is Firebase Authentication.

<https://www.geeksforgeeks.org/what-is-firebase-authentication/>

*Gen Z demand radical change from the food sector to tackle access to healthy and affordable*

*food*. (n.d.). EIT Food. Retrieved March 10, 2025, from

<https://www.eitfood.eu/news/gen-z-demand-radical-change-from-the-food-sector-to-tackle-access-to-healthy-and-affordable-food>

*Introduction • Docs • Svelte*. (n.d.). Svelte. Retrieved March 10, 2025, from

<https://svelte.dev/docs/kit/introduction>

*JWT Security Best Practices*. (2024, July 23). Curity. Retrieved March 10, 2025, from

<https://curity.io/resources/learn/jwt-best-practices/>

*Manage User Sessions | Firebase Authentication*. (n.d.). Firebase. Retrieved March 10, 2025,

from <https://firebase.google.com/docs/auth/admin/manage-sessions>

Novak, S. (2025, February 22). *Mindful Eating Could Be More Beneficial Than Calorie Counting*.

Discover Magazine. Retrieved March 10, 2025, from

<https://www.discovermagazine.com/health/mindful-eating-could-be-more-beneficial-than-calorie-counting>

*Password sign-in best practices*. (2020, October 29). The Firebase Blog. Retrieved March 10,

2025, from <https://firebase.blog/posts/2020/10/password-sign-in-best-practices/>

*Privacy and Security in Firebase*. (n.d.). Firebase. Retrieved March 10, 2025, from

<https://firebase.google.com/support/privacy>

*Safe and secure: The most important facts about content filtering*. (2024, May 14). DriveLock.

Retrieved March 10, 2025, from <https://www.drivelock.com/en/blog/content-filtering>

Sanity. (2024, August 23). *SvelteKit overview*. Sanity. <https://www.sanity.io/glossary/sveltekit>

- Singh, J., & Zahid, T. (2025, March 6). *Reddit unveils new content moderation and analytics tools to boost user engagement*. Reuters. Retrieved March 10, 2025, from <https://www.reuters.com/technology/reddit-unveils-new-content-moderation-analytics-tool-s-boost-user-engagement-2025-03-06/>
- Trifonova, V. (n.d.). *#Foodporn: Food Trends & Insights on Social Media*. GWI. Retrieved March 10, 2025, from <https://www.gwi.com/blog/food-trends-on-social-media>
- Ultra-processed food raises risk of heart disease, heart attack and stroke*. (2023, August 28). British Heart Foundation. Retrieved March 10, 2025, from <https://www.bhf.org.uk/what-we-do/news-from-the-bhf/news-archive/2023/august/ultra-processed-foods-linked-to-cardiovascular-risk>
- Taiwo, I. (2024, December 23). *Web Design Statistics: The Complete Data-Driven Guide for 2024*. Expert Beacon. <https://expertbeacon.com/web-design-statistics/>
- Casner, C. (2025, February 14). *Strawberry-Chocolate Greek Yogurt Bark - Dessert*. EatingWell. Retrieved March 10, 2025, from <https://www.eatingwell.com/recipe/259080/strawberry-chocolate-greek-yogurt-bark/>
- Theoktisto, A. (n.d.). *Chickpea Salad Sandwich*. EatingWell. Retrieved March 10, 2025, from <https://www.eatingwell.com/recipe/7923367/chickpea-salad-sandwich/>
- Peartree, M. (2023, September 19). *Jerk-Spiced Salmon & Quinoa Bowl with Mango Vinaigrette*. EatingWell. Retrieved March 10, 2025, from <https://www.eatingwell.com/recipe/7886983/jerk-spiced-salmon-quinoa-bowl-with-mango-vinaigrette/>
- Collins, E. (2022, June 8). *Banana Oat Egg Pancakes (3 Ingredients!)*. Meaningful Eats. Retrieved March 10, 2025, from <https://meaningfuleats.com/banana-egg-almond-butter-pancakes-grain-free-dairy-free/>
- 57 Healthy Prawn Recipes*. (n.d.). BBC Good Food. Retrieved March 10, 2025, from <https://www.bbcgoodfood.com/recipes/collection/healthy-prawn-recipes>
- Karadsheh, S. (2020, January 21). *Healthy Egg Salad, Mediterranean-Style*. The Mediterranean Dish. Retrieved March 10, 2025, from <https://www.themediterraneandish.com/healthy-egg-salad-mediterranean/>



Diimig, C. (n.d.). *Skillet Steak with Mushroom Sauce*. EatingWell. Retrieved March 10, 2025, from <https://www.eatingwell.com/recipe/266579/skillet-steak-with-mushroom-sauce/>

Schaltegger, M. (2024, March 8). *The KitchenAid Mixer That Cuts 'Baking Time in Half' Is Selling Out Fast, and It's \$139 Off*. Food & Wine. Retrieved March 10, 2025, from <https://www.foodandwine.com/qvc-kitchenaid-standing-mixer-sale-march-2024-8606080>

*Person Icon Images – Browse 7517919 Stock Photos, Vectors, and Video*. (n.d.). Adobe Stock. Retrieved March 10, 2025, from <https://stock.adobe.com/search?k=person+icon>

*Social Media icons for free download*. (n.d.). Freepik. Retrieved March 10, 2025, from <https://www.freepik.com/icons/social-media>